DATA STRUCTURES · DIGITAL NOTES

UNIT I

Introduction to Linear Data Structures

Linear data structures · ADTs · Complexity analysis · Searching · Sorting

Roadmap of this unit

1.1	Linear data structures: definition & importance
1.2	Abstract Data Types (ADTs) & implementation
1.3	Time and space complexity analysis
1.4	Searching: Linear search & Binary search
1.5	Sorting: Bubble, Selection & Insertion sort

1.1 Linear Data Structures

DEFINITION

Data structure = a way of **organising, storing and managing data** in memory so that it can be used **efficiently**.

Linear data structure = a data structure in which elements are arranged **one after another in a sequence**. Every element has exactly one **predecessor** (except the first) and one **successor** (except the last).

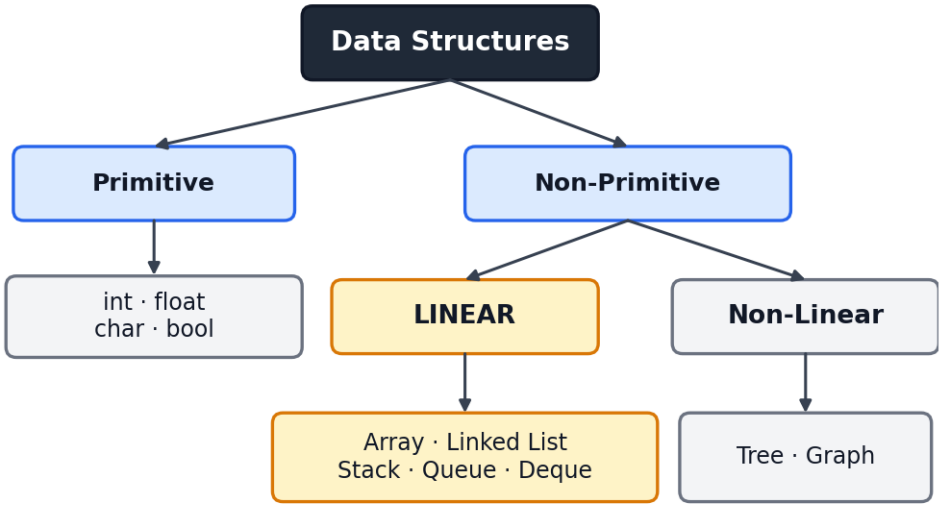


Fig: Classification of data structures – the highlighted branch is the focus of this subject

Why are linear data structures important?

- **Foundation** – every advanced structure (trees, graphs, hash tables) is built using arrays or linked lists.
- **Simple & natural** – real-life data is sequential: a queue at a ticket counter, a stack of plates, a playlist.
- **Easy traversal** – visit all elements in one pass, from first to last.
- **Efficient memory use** – choose static (array) or dynamic (linked) allocation as the problem needs.
- **Used everywhere** – OS scheduling, undo/redo, browser history, music players, expression evaluation.

Characteristics (Properties)

- Elements are stored in a **linear order** (single level).
- Only **one element** can be reached directly after another – traversal is **sequential**.
- Can be implemented in **two ways**: **contiguous memory** (arrays) or **linked memory** (pointers).
- Operations: **insert, delete, traverse, search, sort**.

Array – the basic linear structure

An **array** is a collection of **same-type** elements stored in **contiguous memory locations**, accessed using an **index**.

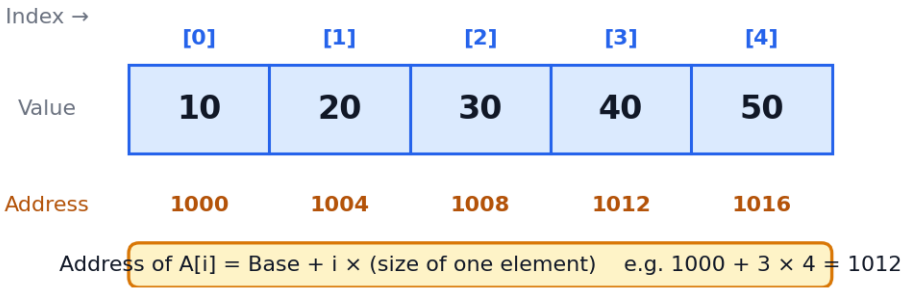


Fig: Array stored in contiguous memory – address calculation for any index is instant

Feature	Linear	Non-linear
Arrangement	Sequential, one level	Hierarchical / network, many levels
Traversal	Single run (one pass)	Needs multiple paths / recursion
Relationship	One-to-one	One-to-many or many-to-many
Memory use	Simple, predictable	More complex
Examples	Array, List, Stack, Queue	Tree, Graph

Basic operations on a linear structure

Operation	Meaning	Array cost
Traversal	Visit every element once	O(n)
Access	Read / write element at index i	O(1)
Insertion	Add an element at a position (shift right)	O(n)
Deletion	Remove an element (shift left)	O(n)
Searching	Find the location of a value	O(n) / O(log n)
Sorting	Arrange in order	O(n log n) – O(n^2)

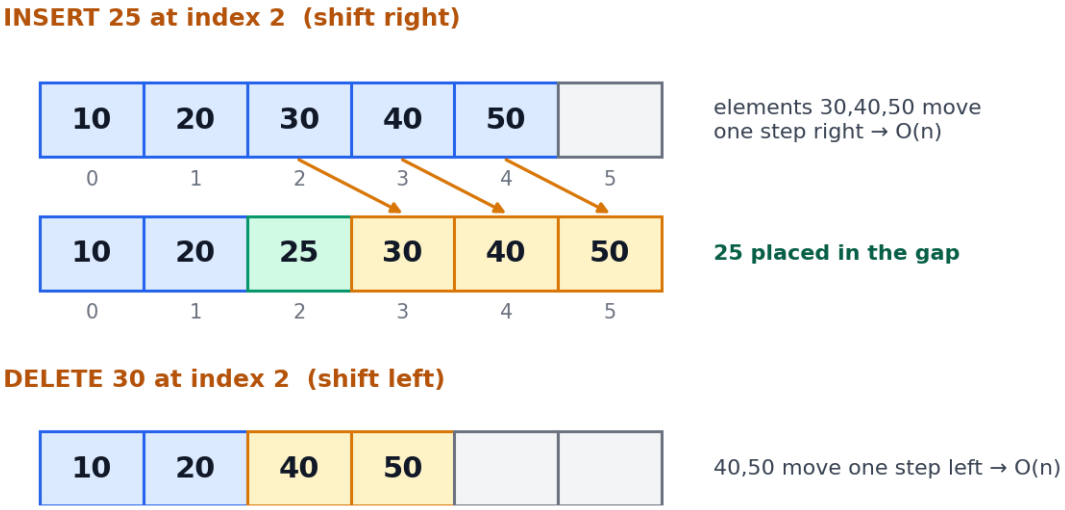


Fig: Insertion and deletion in an array need shifting of elements

REMEMBER

- Array size is **fixed** at creation (static) – cannot grow later.
- Insert/delete in the **middle is costly** because of shifting; access by index is **very fast**.

1.2 Abstract Data Types (ADT)

DEFINITION

ADT = a **logical description** of *what* a data type stores and *what operations* can be done on it – without saying *how* they are implemented.

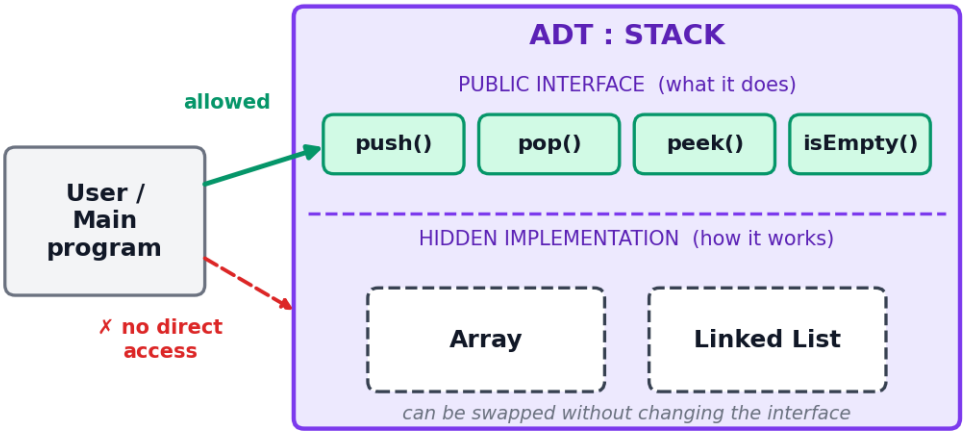


Fig: ADT block diagram – the user sees only the interface; the implementation stays hidden

Key points

- An ADT = **Data** (values it holds) + **Operations** (what you can do) + **Rules** (behaviour, e.g. LIFO).
- **Abstraction** – show the essential features, hide the details.
- **Encapsulation** – data and its operations are bundled; data is accessed only through the interface.
- Same ADT can have **many implementations** (Stack → array or linked list).

ADT	Data stored	Main operations	Rule
List	Sequence of items	insert, delete, search, traverse	Order maintained
Stack	Items in a pile	push, pop, peek, isEmpty	LIFO
Queue	Items in a line	enqueue, dequeue, front	FIFO

ADT vs Data Structure

ADT	Data Structure
Logical view – *what*	Physical view – *how*
Defined by operations	Defined by memory layout & code
Language independent	Written in a language (C, Java…)
Example: Stack ADT	Example: stack using array

Implementing an ADT (example: Stack in C)

```
#define MAX 100
typedef struct {            // DATA part
    int items[MAX];
    int top;
} Stack;

void init(Stack *s)           { s->top = -1; }                // OPERATIONS part
int  isEmpty(Stack *s)        { return s->top == -1; }
void push(Stack *s, int x)    { s->items[++(s->top)] = x; }
int  pop(Stack *s)            { return s->items[(s->top)--]; }
```

The user calls push() / pop() only – never touches items[] or top directly

Advantages of using ADTs

- **Reusability** – write once, use in many programs.
- **Easy maintenance** – change the implementation without touching user code.
- **Security** – internal data cannot be corrupted from outside.
- **Clarity** – programmers think in terms of operations, not memory.

REMEMBER

Steps to implement an ADT: ① define the data → ② define the operations → ③ choose the structure (array / linked list) → ④ write the functions → ⑤ test.

1.3 Time and Space Complexity

DEFINITIONS

- **Algorithm** = a finite, step-by-step procedure to solve a problem.
- **Time complexity** = how the **running time grows** as the input size **n** grows (count of basic steps, *not* seconds).
- **Space complexity** = how the **extra memory** needed grows with n (input space + auxiliary space).

Asymptotic notations

Notation	Meaning	Used for
Big-O O( )	**Upper bound** – never slower than this	Worst case (most common)
Omega Ω( )	**Lower bound** – never faster than this	Best case
Theta Θ( )	**Tight bound** – upper and lower both match	Average / exact growth

Best, average and worst case

- **Best case** – input that makes the algorithm fastest (e.g., key is the first element).
- **Average case** – typical input.
- **Worst case** – input that makes it slowest (e.g., key is absent). **We usually report this.**

How to count steps

- Simple statement → **1 step** → O(1)
- Single loop running n times → **n steps** → O(n)
- Loop inside a loop (both n) → **n × n** → O(n^2)
- Loop where the counter doubles/halves (i = i×2) → O(log n)
- Sequential blocks → **add** and keep the largest term. **Drop constants:** 5n + 3 → O(n)

```
for (i = 1; i <= n; i++)          // runs n times
    for (j = 1; j <= n; j++)      // runs n times for each i
        count++;                  // executes n x n times  =>  O(n^2)

for (i = 1; i < n; i = i * 2)     // i = 1,2,4,8...  =>  log2(n) times  =>  O(log n)
    count++;
```

Two examples of counting steps

Worked example – total time of a function

```
int sumArray(int a[], int n) {
    int sum = 0;                  // 1 step
    for (int i = 0; i < n; i++)   // loop runs n times
        sum += a[i];              // n steps
    return sum;                   // 1 step
}                                 // T(n) = n + 2   =>  O(n)     Space: sum, i  =>  O(1)
```

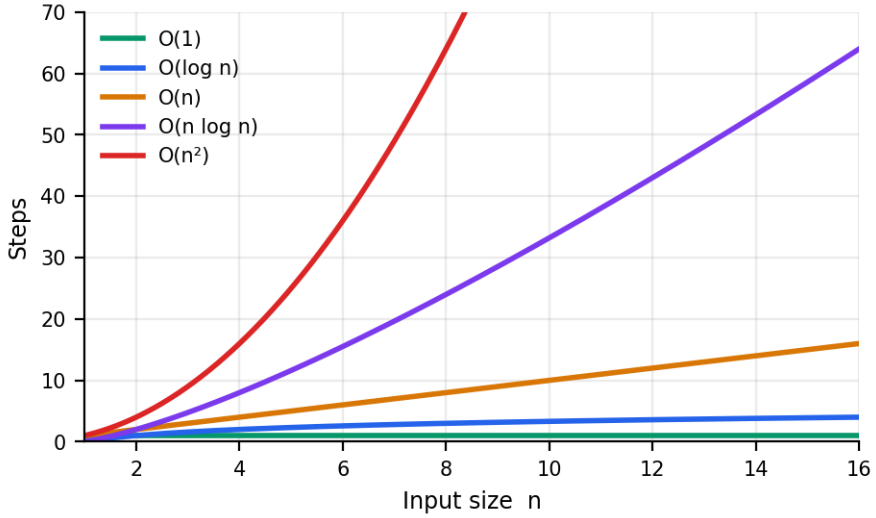


Fig: Growth of common complexities – smaller curve = faster algorithm

Order	Name	Example
O(1)	Constant	Array access a[i]
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, traversal
O(n log n)	Linearithmic	Merge sort
O(n^2)	Quadratic	Bubble, Selection, Insertion sort

Space complexity

- **In-place** algorithm uses only a few extra variables → **O(1) space** (bubble, selection, insertion sort).
- Algorithm that creates a copy of the input uses **O(n) space**.
- Recursion uses the call stack → recursion of depth n costs O(n) space.

EXAM TIP
Order to remember: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$. Always write complexity as a **function of n**.

1.4 Searching Techniques

DEFINITION
Searching = finding the location of a given element (**key**) in a collection. Result: **successful** (index returned) or **unsuccessful** (e.g., −1).

A. Linear (Sequential) Search

- Compare the key with each element **one by one**, from the first to the last.
- Works on **sorted or unsorted** data.
- Simple but slow for large data.

Algorithm

1. Start from index **i = 0**.
2. If **a[i] == key** → return i (found).
3. Else **i = i + 1**; repeat until i = n.
4. If the loop ends → return −1 (not found).



Fig: Linear search for key 35 – found in the 3rd comparison

```
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;     // found
    return -1;                         // not found
}
```

Case	Time	When
Best	O(1)	Key is at the first position
Average	O(n)	Key in the middle
Worst	O(n)	Key is last or absent
Space	O(1)	No extra memory

EXAM TIP

- Linear search is also the **only** choice for an **unsorted linked list**.
- If the key occurs many times, the basic version returns the **first** match.

B. Binary Search

COMMON MISTAKE

Binary search works **only on a SORTED array**. On unsorted data the answer will be wrong.

- Uses **divide and conquer**: compare the key with the **middle** element and discard half of the array each time.
- **mid = low + (high − low) / 2**
- If key = a[mid] → found. If key > a[mid] → **low = mid + 1**. If key < a[mid] → **high = mid − 1**.

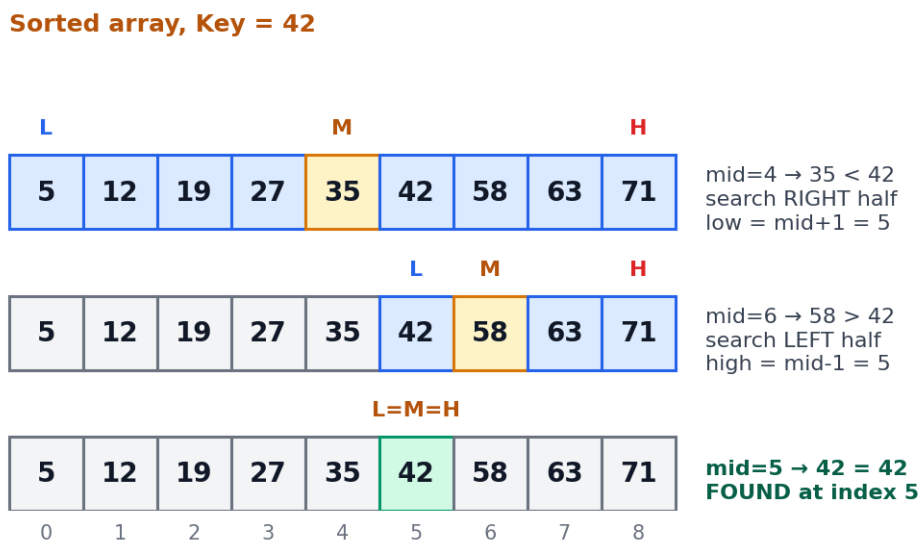


Fig: Binary search for key 42 – only 3 comparisons for 9 elements

```
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key)      return mid;      // found
        else if (a[mid] < key)  low  = mid + 1;  // go right
        else                    high = mid - 1;  // go left
    }
    return -1;                                   // not found
}
```

- Each step halves the data: n → n/2 → n/4 … → 1. Steps = **$\log_2 n$**. For 1,00,000 elements → only **≈ 17** comparisons.

Basis	Linear Search	Binary Search
Data must be sorted?	No	**Yes**
Technique	Sequential scan	Divide and conquer
Best / Worst time	O(1) / O(n)	O(1) / O(log n)
Works on	Array, linked list	Array (random access needed)
Best for	Small or unsorted data	Large sorted data

1.5 Sorting Techniques

DEFINITION
Sorting = arranging elements in **ascending** or **descending** order.

- **Stable sort** – equal elements keep their original relative order.
- **In-place sort** – needs only O(1) extra memory.
- **Pass** – one complete round of the outer loop.
- Sorted data makes searching (binary search) very fast.

A. Bubble Sort

- Repeatedly **compares adjacent elements** and **swaps** them if they are in the wrong order.
- After pass 1 the **largest** element "bubbles up" to the last position; after pass 2 the second largest, and so on.
- **Optimisation:** if a pass makes **no swap**, stop – the array is already sorted.



Fig: Bubble sort on [5, 1, 4, 2, 8] – green = fixed in final position

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {                // wrong order -> swap
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;                       // already sorted
    }
}
```

B. Selection Sort

- Find the **minimum** element in the unsorted part and **swap** it with the first unsorted position.
- The sorted part grows from the **left** by one element every pass.
- Only **n − 1 swaps** in total – good when swapping is costly.



Fig: Selection sort on [64, 25, 12, 22, 11] – amber = position i, red = minimum found

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;            // remember smallest index
        int t = a[i]; a[i] = a[min]; a[min] = t;   // one swap per pass
    }
}
```

C. Insertion Sort

- Works like **arranging playing cards** in your hand.
- Take the next element (**key**) and **insert** it at its correct place in the already-sorted left part by **shifting** bigger elements right.
- Very fast on **nearly sorted** data – best case O(n).



Fig: Insertion sort on [7, 4, 5, 2] – green = sorted part, amber = key being inserted

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {   // shift larger elements right
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                  // place the key
    }
}
```

Comparison of sorting techniques

Algorithm	Best	Average	Worst	Space	Stable?	Remark
Bubble	O(n)*	$O(n^2)$	$O(n^2)$	O(1)	Yes	Simple; many swaps
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Fewest swaps
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Best for nearly sorted data

* with the “no swap” optimisation.

Which sort should I use?

Situation	Best choice	Reason
Data is nearly sorted	Insertion sort	Very few shifts → close to O(n)
Writes / swaps are expensive	Selection sort	Only n − 1 swaps
Very small data or teaching	Bubble sort	Easiest to code and understand
Large data	Merge / Quick sort	O(n log n) – beyond this unit

Applications of searching and sorting

- **Searching:** finding a contact in a phone book, looking up a student by roll number, checking a login name.
- **Sorting:** ranking exam results, ordering products by price, arranging files by date, preparing data for binary search.

Try it yourself (dry-run practice)

1. Apply **bubble sort** to 9, 3, 7, 1 and write the array after each pass.
2. Apply **selection sort** to 29, 10, 14, 37, 13 and list every swap.
3. Apply **insertion sort** to 8, 5, 6, 2 – show the shifts for each key.
4. Search key **27** in [3, 9, 14, 21, 27, 33, 40] using binary search – write low, mid, high for every step.
5. How many comparisons does binary search need in the worst case for n = 1024? *(Answer: 11)*

1.6 Quick Revision & Practice

ONE-PAGE SUMMARY

- Linear DS → elements in a **sequence** (array, linked list, stack, queue).
- ADT → **what** (operations) not **how** (implementation).
- Time complexity → growth with n; use **Big-O**; drop constants.
- Linear search O(n) – any data. Binary search O(log n) – **sorted** data only.
- Bubble = swap neighbours · Selection = pick minimum · Insertion = place into sorted part.
- All three simple sorts: worst case **O(n²)**, space **O(1)**.

Likely exam questions

2 marks

- Define a linear data structure. Give two examples.
- What is an ADT?
- Define time complexity and space complexity.
- Why does binary search need sorted data?
- What is a stable sort?

5 marks

- Differentiate between linear and non-linear data structures.
- Explain Big-O, Omega and Theta notations.
- Write the algorithm for linear search and analyse its complexity.
- Trace insertion sort on 9, 5, 1, 4, 3.

10 marks

- Explain binary search with an example and derive its time complexity.
- Explain bubble sort, selection sort and insertion sort with programs and compare them.
- Explain ADT with the help of a Stack ADT example.

DATA STRUCTURES · DIGITAL NOTES

UNIT II

Linked Lists

Singly · Doubly · Circular linked lists · Arrays vs linked lists · Applications

Roadmap of this unit

2.1	Introduction – why linked lists?
2.2	Singly linked list: representation & operations
2.3	Doubly linked list
2.4	Circular linked list
2.5	Comparing arrays and linked lists
2.6	Applications of linked lists

2.1 Introduction to Linked Lists

DEFINITION
Linked list = a linear data structure made of **nodes**. Each node stores **data** and a **link (pointer)** to the next node. Nodes can lie **anywhere in memory**; the links join them in order.

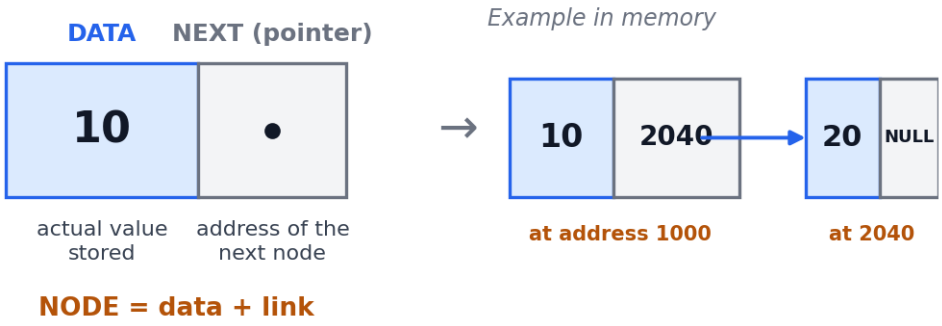


Fig: Structure of a node – data part and pointer part

Why do we need linked lists? (problems of arrays)

- Array size is **fixed** – wastes memory if too big, overflows if too small.
- Insert / delete in the middle needs **shifting** of many elements – O(n).
- Needs a **big continuous** block of free memory.
- Linked list: size **grows and shrinks at run time** (dynamic) and insertion needs only pointer changes.

Terminology

Term	Meaning
Node	One element = data + pointer(s)
head / start	Pointer to the **first** node – the only way to enter the list
next	Pointer to the following node
NULL	Special value meaning “no node” – marks the end
Traversal	Visiting each node by following next links

Types of linked lists

Type	Links per node	Direction	Last node points to
Singly	1 (next)	Forward only	NULL
Doubly	2 (prev, next)	Forward + backward	NULL
Circular	1 or 2	Forward (or both)	**First node**

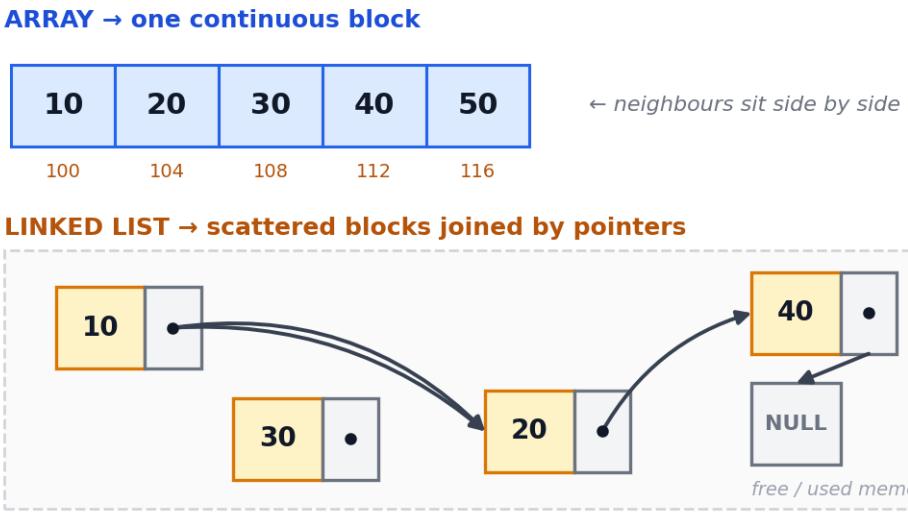


Fig: Array = continuous memory; linked list = scattered nodes connected by pointers

REMEMBER

- **Advantages:** dynamic size · easy insertion/deletion · no memory shifting · efficient memory use.
- **Disadvantages:** extra memory for pointers · **no direct access** (no a[i]) · sequential traversal only · poor cache use.

2.2 Singly Linked List (SLL)

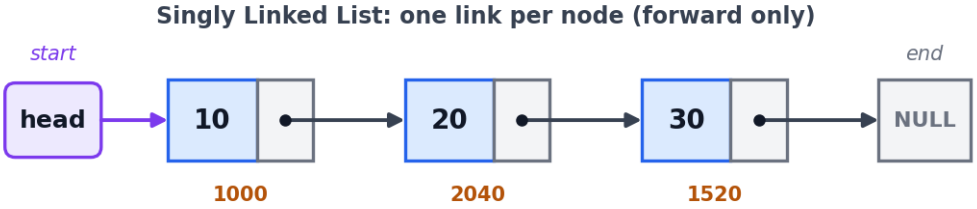


Fig: Singly linked list with addresses – head holds the address of the first node

Representation in C

```
struct Node {
    int data;              // value stored in the node
    struct Node *next;     // address of the next node (self-referential structure)
};
struct Node *head = NULL;  // empty list

struct Node* createNode(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));   // allocate
    n->data = x;  n->next = NULL;
    return n;
}
```

A node is a self-referential structure – it contains a pointer to its own type

Operation	Time	Remark
Insert at beginning	O(1)	No traversal needed
Insert at end	O(n)	O(1) if a tail pointer is kept
Insert after a given node	O(1)	Position already known
Delete first node	O(1)	Move head
Delete last node	O(n)	Need the second-last node
Search / access k-th	O(n)	Must walk from head

A. Traversal (display)

1. Set **temp = head**.
2. While **temp != NULL**: print temp→data, then **temp = temp→next**.
3. Stop when temp becomes NULL.

```
void display() {
    struct Node *temp = head;
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL\n");
}
```

B. Insertion

1) At the beginning

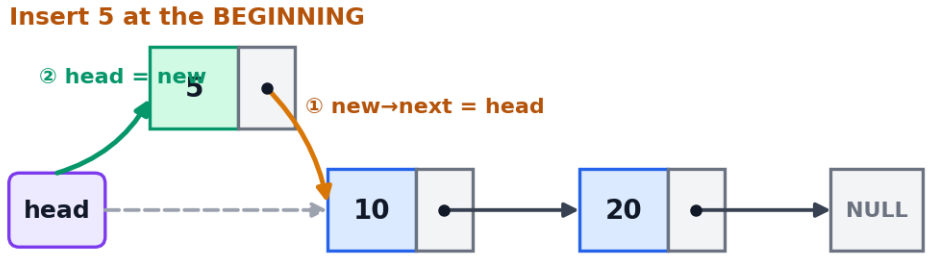


Fig: Insertion at the beginning – only two pointer changes

```
void insertBegin(int x) {
    struct Node *n = createNode(x);
    n->next = head;      // (1) new node points to old first node
    head = n;            // (2) head now points to new node
}
```

2) In the middle (after a given position)

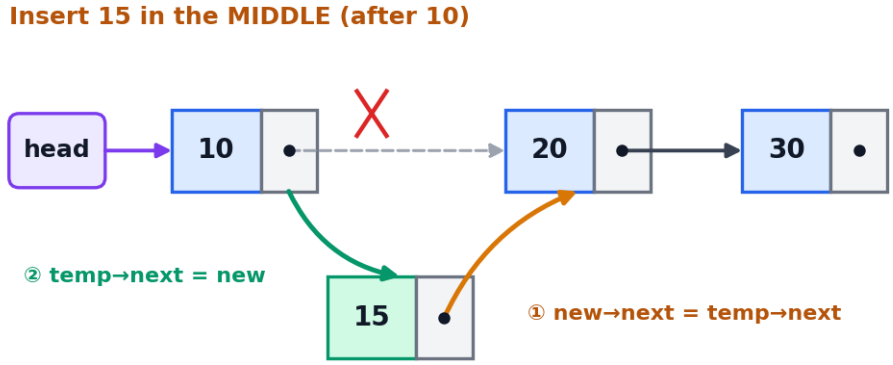


Fig: Insertion after node 10 – order of the two steps matters!

COMMON MISTAKE
Do **step ① first** (new→next = temp→next). If you do ② first, you lose the address of the rest of the list.

3) At the end

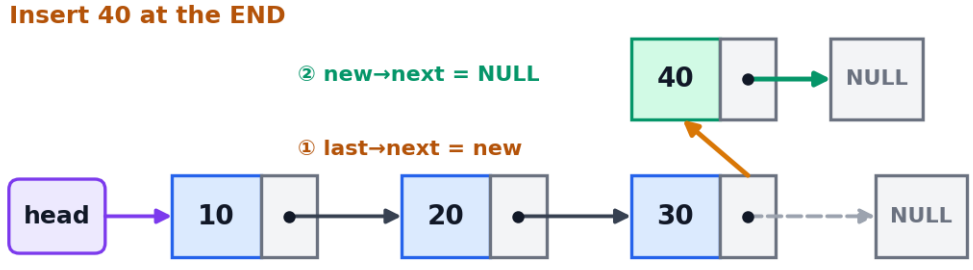


Fig: Insertion at the end – walk to the last node, then link

```
void insertEnd(int x) {
    struct Node *n = createNode(x), *temp = head;
    if (head == NULL) { head = n; return; }      // empty list
    while (temp->next != NULL) temp = temp->next; // go to the last node
    temp->next = n;                               // link last node to new node
}

void insertAfter(struct Node *p, int x) {        // insert after node p
    if (p == NULL) return;
    struct Node *n = createNode(x);
    n->next = p->next;     // step 1
    p->next = n;           // step 2
}
```

C. Deletion

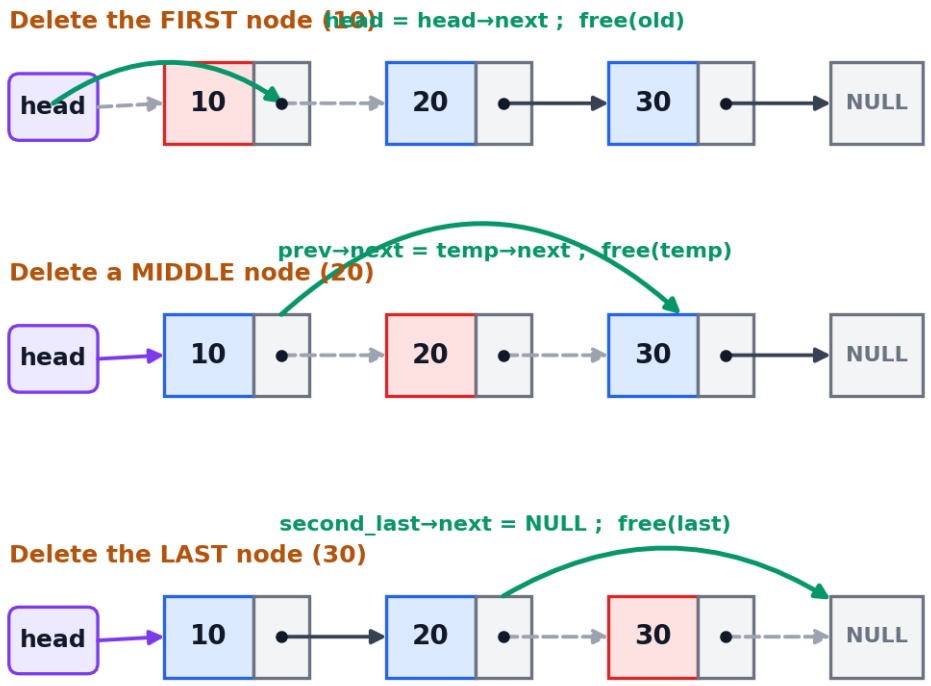


Fig: Deleting the first, a middle and the last node – dashed arrows are removed links

```
void deleteBegin() {
    if (head == NULL) { printf("Underflow"); return; }
    struct Node *t = head;  head = head->next;  free(t);
}
void deleteKey(int key) {                         // delete first node having value key
    struct Node *t = head, *prev = NULL;
    while (t != NULL && t->data != key) { prev = t; t = t->next; }
    if (t == NULL) return;                        // key not found
    if (prev == NULL) head = t->next;             // deleting the head
    else prev->next = t->next;                    // bypass the node
    free(t);
}
```

free() returns memory to the system – forgetting it causes a memory leak

D. Searching

```
int search(int key) {
    struct Node *t = head; int pos = 1;
    while (t != NULL) { if (t->data == key) return pos; t = t->next; pos++; }
    return -1;                    // not found      Time: O(n)
}
```

Worked example – tracing a list step by step

Step	Operation	List after the operation
1	insertBegin(30)	head → 30 → NULL
2	insertBegin(20)	head → 20 → 30 → NULL
3	insertEnd(40)	head → 20 → 30 → 40 → NULL
4	insertAfter(node 20, 25)	head → 20 → 25 → 30 → 40 → NULL
5	deleteKey(30)	head → 20 → 25 → 40 → NULL
6	deleteBegin()	head → 25 → 40 → NULL

E. Reversing a singly linked list

- Reverse the **direction of every link** – no data is moved.
- Use **three pointers**: **prev** (already reversed part), **curr** (node being handled), **next** (saved rest of list).
- Time **O(n)**, extra space **O(1)**.

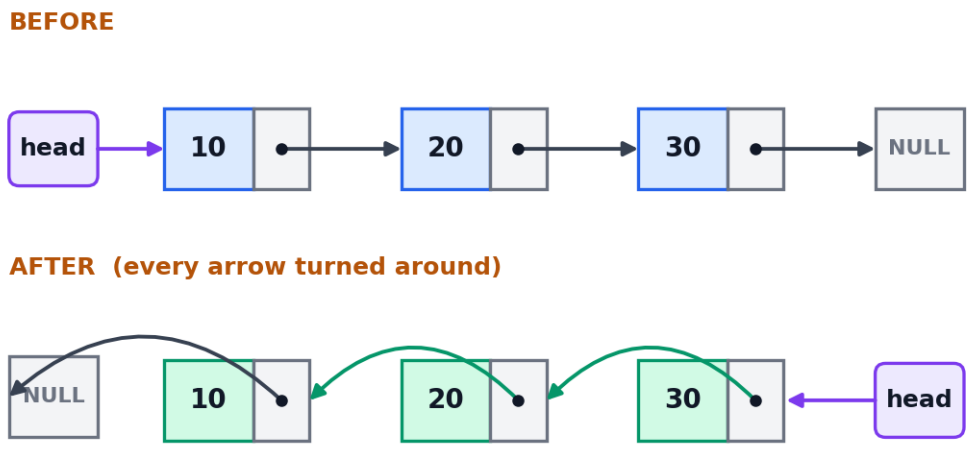


Fig: Reversing a list – last node becomes the new head

```
void reverse() {
    struct Node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next;     // 1. save the rest of the list
        curr->next = prev;     // 2. turn the arrow around
        prev = curr;           // 3. move prev forward
        curr = next;           // 4. move curr forward
    }
    head = prev;               // prev is now the first node
}
```

COMMON MISTAKES

- Losing the **head** pointer = losing the whole list.
- Dereferencing a **NULL** pointer (e.g., temp→next when temp is NULL) crashes the program.
- Forgetting to **free()** deleted nodes causes memory leaks.
- Changing pointers in the **wrong order** disconnects the list.

Useful variants

- **Tail pointer** – keep a second pointer to the last node → insertion at the end becomes O(1).
- **Dummy (header) node** – a fake first node removes special cases for inserting / deleting at the head.
- **Finding the middle** – move **slow** by 1 and **fast** by 2; when fast reaches the end, slow is at the middle.

2.3 Doubly Linked List (DLL)

DEFINITION
Doubly linked list = each node has **three parts**: **prev** pointer, **data**, **next** pointer. It can be traversed in **both directions**.

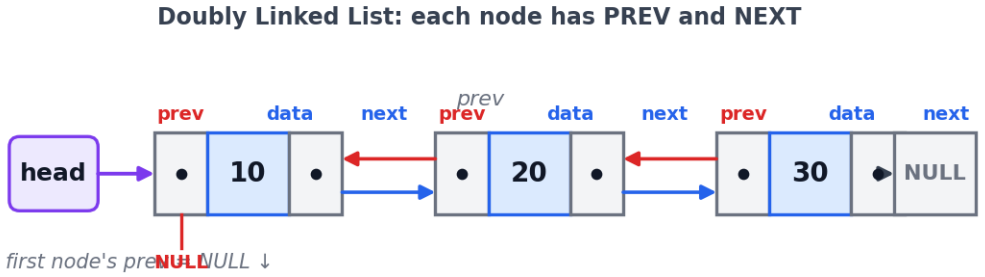


Fig: Doubly linked list – first node's prev and last node's next are NULL

```
struct DNode {
    struct DNode *prev;
    int data;
    struct DNode *next;
};
```

Advantages over singly linked list

- **Backward traversal** is possible.
- A node can be **deleted without a separate prev pointer** – the node itself knows its predecessor.
- Insertion / deletion before a given node is easy.

Disadvantages

- Extra memory for the **prev** pointer in every node.
- More pointers to update → **more chance of mistakes**.

Traversal in both directions

```
void forward(struct DNode *head) {                // first -> last
    for (struct DNode *t = head; t != NULL; t = t->next) printf("%d ", t->data);
}
void backward(struct DNode *tail) {               // last -> first (needs a tail pointer)
    for (struct DNode *t = tail; t != NULL; t = t->prev) printf("%d ", t->data);
}
```

Insertion between two nodes A and B

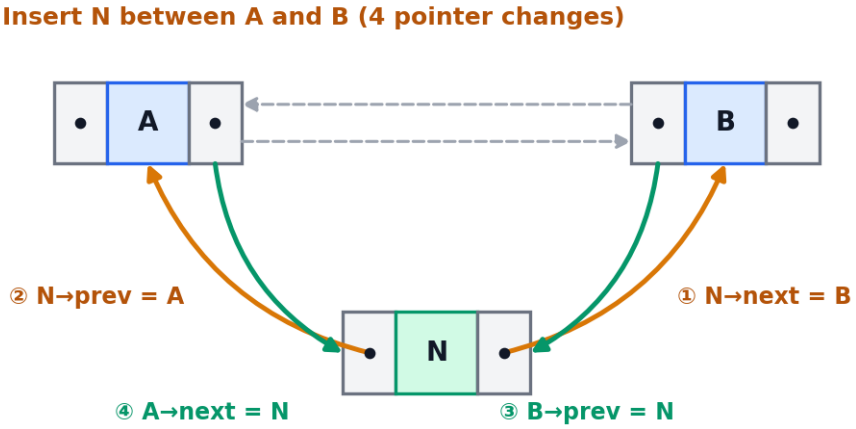


Fig: Four pointer changes are needed in a doubly linked list

```
void insertAfter(struct DNode *A, int x) {          // insert new node N after A
    struct DNode *N = (struct DNode*) malloc(sizeof(struct DNode));
    N->data = x;
    N->next = A->next;              // (1) N -> B
    N->prev = A;                    // (2) N <- A
    if (A->next != NULL) A->next->prev = N;   // (3) B -> prev = N
    A->next = N;                    // (4) A -> next = N
}
```

Deletion of a node X

1. **X→prev→next = X→next** (previous node skips X)
2. **X→next→prev = X→prev** (next node skips X) – only if X→next exists
3. **free(X)**

EXAM TIP
Special cases to mention in exams: deleting the **first** node (update head) and the **last** node (X→next is NULL). Always check for an **empty list**.

2.4 Circular Linked List (CLL)

DEFINITION
Circular linked list = a linked list in which the **last node points back to the first node**, so there is **no NULL** at the end.

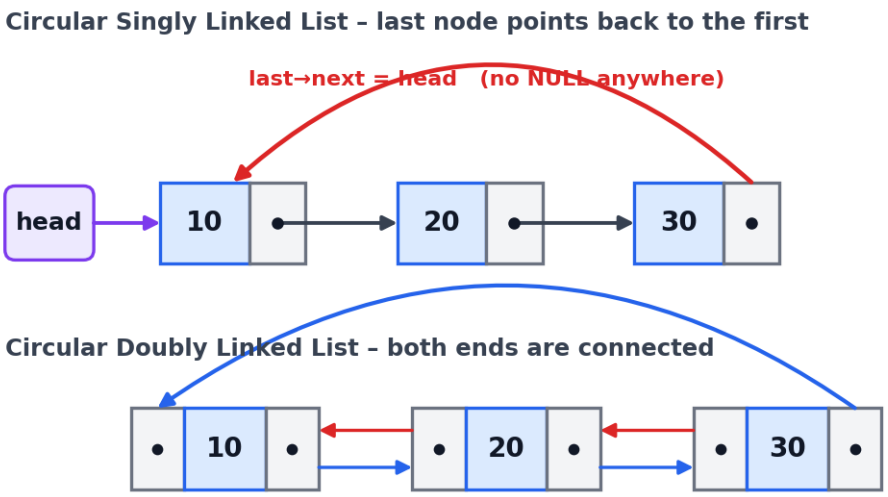


Fig: Circular singly and circular doubly linked lists

Key points

- Can be **singly** or **doubly** circular.
- From **any node** you can reach every other node.
- Traversal stops when we come **back to the starting node** – NOT at NULL.
- If a pointer **tail** to the last node is kept, insertion at both ends is **O(1)** (head = tail→next).
- Danger: using “while (temp != NULL)” gives an **infinite loop**.

```
void displayCircular() {
    if (head == NULL) return;
    struct Node *t = head;
    do {
        printf("%d ", t->data);
        t = t->next;
    } while (t != head);          // stop when we are back at the start
}

void insertEndCircular(int x) {   // singly circular
    struct Node *n = createNode(x);
    if (head == NULL) { head = n; n->next = head; return; }
    struct Node *t = head;
    while (t->next != head) t = t->next;   // find last node
    t->next = n;   n->next = head;         // keep the circle closed
}
```

Deletion in a circular list (singly)

- **Only one node:** free it and set head = NULL.
- **Deleting the first node:** find the last node, set last→next = head→next, move head, free the old head.
- **Deleting any other node:** same as in a singly list – prev→next = temp→next.

Basis	Singly	Doubly	Circular
Pointers / node	1	2	1 or 2
Traversal	Forward	Both ways	Round and round
End marker	NULL	NULL	Link to first node
Extra memory	Least	Most	Same as base type
Best for	Simple dynamic data	Back / forward moves	Repeating cycles

2.5 Arrays vs Linked Lists

Basis	Array	Linked list
Memory allocation	Static (compile time), fixed size	Dynamic (run time), grows / shrinks
Memory layout	Contiguous	Scattered, joined by pointers
Access of element	**Direct** a[i] → O(1)	Sequential → O(n)
Insertion / deletion	Costly – shifting, O(n)	**Easy** – O(1) once position is known
Extra memory	None	Pointer(s) in every node
Memory wastage	Possible (unused slots)	No unused slots, but pointer overhead
Searching	Binary search possible (sorted)	Only linear search
Cache performance	Good (neighbouring data)	Poor
Use when	Size known, frequent random access	Size unknown, frequent insert / delete

KEY IDEA
Rule of thumb: need quick **access by index** → choose an **array**. Need frequent **insertion / deletion** and the size keeps changing → choose a **linked list**.

2.6 Applications of Linked Lists



Fig: Typical applications grouped by type of list

Polynomial representation

Each term is stored as a node: **coefficient | exponent | next**. Adding two polynomials = merge the two lists by exponent. No space is wasted for missing terms (sparse polynomial).

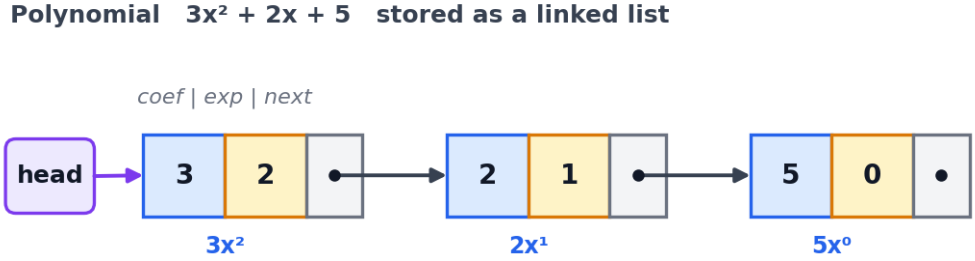


Fig: Polynomial 3x² + 2x + 5 using a linked list

Example – round-robin CPU scheduling (circular list)

Processes P1 → P2 → P3 form a circle. The CPU gives each process a **time slice** and then moves to the next node. A finished process is deleted from the circle; the loop continues until the list is empty.

Time slice	CPU runs	Circle after the slice
1	P1	P1 → P2 → P3 → (P1)
2	P2 (finishes)	P1 → P3 → (P1)
3	P3	P1 → P3 → (P1)
4	P1 (finishes)	P3 → (P3)

More applications

- **Implementation of stacks and queues** (dynamic, no overflow until memory ends).
- **Dynamic memory management** – operating system keeps a linked list of free blocks.
- **Hash table chaining** – colliding keys are stored in a list (Unit V).
- **Graph adjacency lists** – list of neighbours of each vertex.
- **Undo / redo** and **browser history** – doubly linked list of states / pages.
- **Music or image playlist** – next / previous (DLL) and repeat-all (circular list).
- **Round-robin scheduling** – processes arranged in a circular list; CPU moves to the next after every time slice.
- **Sparse matrix** and **big-number arithmetic** – store only non-zero parts / digits.

2.7 Quick Revision & Practice

ONE-PAGE SUMMARY

- Linked list = **nodes** (data + pointer) joined by links; head = entry point; NULL = end.
- SLL: insert at beginning O(1); end O(n); delete last O(n); search O(n).
- DLL: prev + next → both directions, 4 pointer changes on insertion.
- CLL: last → first; loop with **do-while (t != head)**.
- Array → fast access, slow insert/delete. Linked list → slow access, fast insert/delete.
- Always **update pointers in the right order** and **free()** deleted nodes.

Likely exam questions

2 marks

- What is a self-referential structure?
- Define a node. What are its parts?
- Mention two advantages of a doubly linked list.
- What is a circular linked list?
- Why is binary search not suitable for linked lists?

5 marks

- Write an algorithm to insert a node at the beginning and at the end of a singly linked list.
- Explain deletion of a node from a doubly linked list with a diagram.
- Compare arrays and linked lists.
- Explain the representation of a polynomial using a linked list.

10 marks

- Explain all operations (create, insert, delete, search, display) on a singly linked list with C code.
- Explain singly, doubly and circular linked lists with diagrams and applications.

Try it yourself

1. Draw the list after: insertBegin(30), insertBegin(20), insertEnd(40), insertAfter(20, 25).
2. Write a function to **count the nodes** of a singly linked list.
3. Write a function to **reverse** a singly linked list using three pointers (prev, curr, next).
4. Find the **middle** node in one pass using two pointers (slow and fast).

DATA STRUCTURES · DIGITAL NOTES

UNIT III

Stacks

LIFO structure · Push & Pop · Array and linked implementation · Expression evaluation · Backtracking

Roadmap of this unit

3.1	Introduction to stacks: properties & terminology
3.2	Stack operations & the Stack ADT
3.3	Implementing a stack using an array
3.4	Implementing a stack using a linked list
3.5	Applications: expressions, parentheses, reversing, backtracking, recursion

3.1 Introduction to Stacks

DEFINITION

Stack = a linear data structure in which **insertion and deletion happen only at one end**, called the **TOP**. It follows the **LIFO** principle – **L**ast **I**n, **F**irst **O**ut: the element pushed last is the first one to be removed.

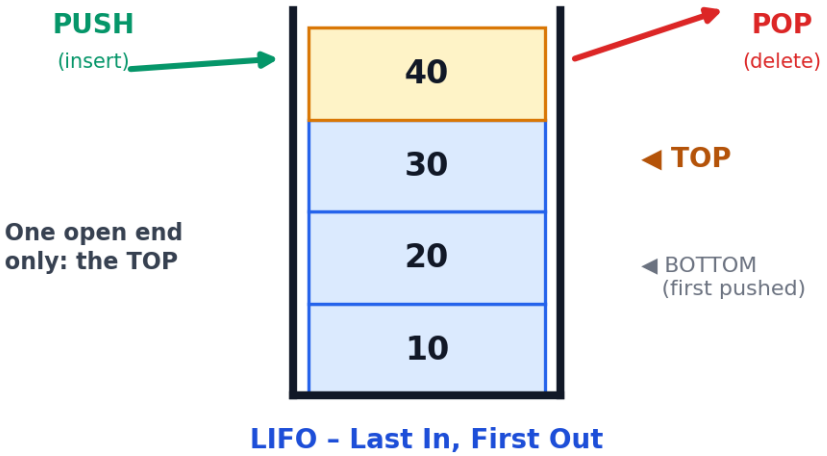


Fig: Stack – a one-ended container; push and pop both work on the TOP

Real-life examples

- A **pile of plates** – you add or take only from the top.
- A **stack of books** or a box of **tennis balls** in a tube.
- **Undo** in a text editor – last change is undone first.
- **Browser Back button** – most recent page opens first.

Properties

- **Linear** and **ordered by time of insertion**.
- **LIFO** (or FILO) access.
- Only the **top** element is accessible – no random access.
- Size can be **fixed** (array) or **dynamic** (linked list).
- Every basic operation takes **O(1)** time.

Terminology

Term	Meaning
TOP	Position of the most recently inserted element (top = −1 means empty stack in array form)
PUSH	Insert an element on the top
POP	Remove and return the top element
PEEK / TOP	Read the top element **without** removing it
Overflow	Trying to push into a **full** stack
Underflow	Trying to pop from an **empty** stack

3.2 Stack Operations and ADT

Operation	Purpose	Time
push(x)	Add x on the top (check overflow first)	O(1)
pop()	Remove top element (check underflow first)	O(1)
peek()	Return top element, stack unchanged	O(1)
isEmpty()	True if stack has no element	O(1)
isFull()	True if no space left (array stack)	O(1)
size()	Number of elements present	O(1)

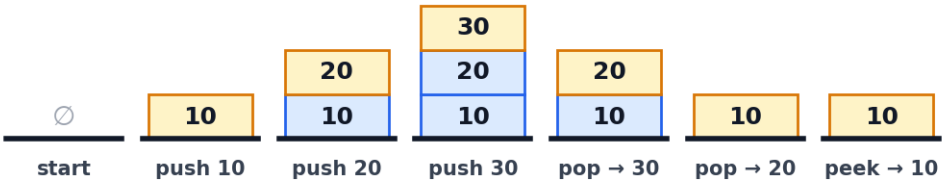


Fig: Sequence of operations – amber cell is always the TOP

Stack ADT specification

Operation	Precondition	Postcondition
push(x)	Stack is not full	x is the new top; size increases by 1
pop()	Stack is not empty	Previous top is removed and returned; size decreases by 1
peek()	Stack is not empty	Top value returned; stack unchanged
isEmpty()	None	Returns true if size = 0

Algorithms

PUSH(x)

1. If **top == MAX − 1** → print “Overflow” and stop.
2. **top = top + 1**
3. **stack[top] = x**

POP()

1. If **top == −1** → print “Underflow” and stop.
2. **x = stack[top]**
3. **top = top − 1**
4. Return x.

Dry run on an array stack (MAX = 3)

Operation	top after	stack[0] [1] [2]	Output / message
(start)	−1	_ _ _	empty
push(10)	0	10 _ _	
push(20)	1	10 20 _	
push(30)	2	10 20 30	stack is now full
push(40)	2	10 20 30	**Overflow**
pop()	1	10 20 (30)	returns 30
pop(), pop()	−1	(10)(20)(30)	returns 20, 10
pop()	−1	_ _ _	**Underflow**

Values in brackets are still in memory but are no longer part of the stack (they are simply overwritten by the next push).

EXAM TIP
In push, **increment first, then store**. In pop, **read first, then decrement**. Always check overflow / underflow **before** changing top.

3.3 Stack using an Array

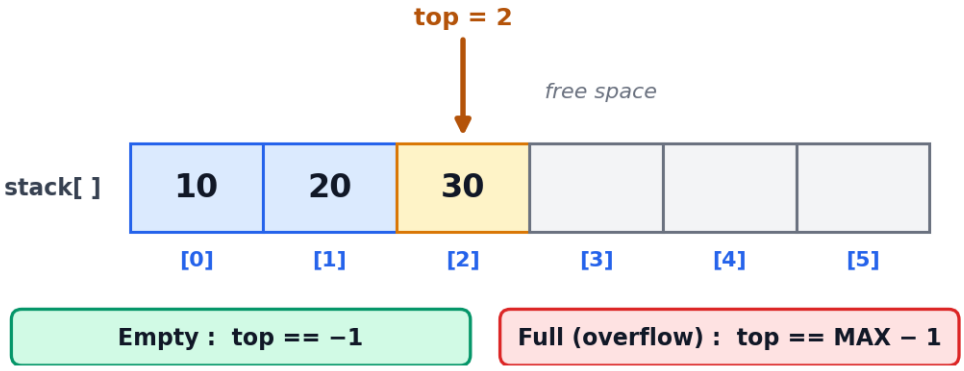


Fig: Array-based stack – top stores the index of the last element

- Use an array **stack[MAX]** and an integer **top** initialised to **−1**.
- Push → **++top** then store. Pop → read then **top−−**.
- Memory is **contiguous**; size is fixed at **MAX**.

```c
#define MAX 5
int stack[MAX];
int top = -1;                                   // empty stack

void push(int x) {
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    stack[++top] = x;                           // increment, then store
}
int pop() {
    if (top == -1) { printf("Stack Underflow\n"); return -1; }
    return stack[top--];                        // read, then decrement
}
int peek()    { return (top == -1) ? -1 : stack[top]; }
int isEmpty() { return top == -1; }
int isFull()  { return top == MAX - 1; }
```

Complete array implementation of a stack

Two stacks in one array

To use space well, two stacks can share **one array**: stack 1 starts at index 0 and grows right; stack 2 starts at the last index and grows left. Overflow happens only when the two tops meet.

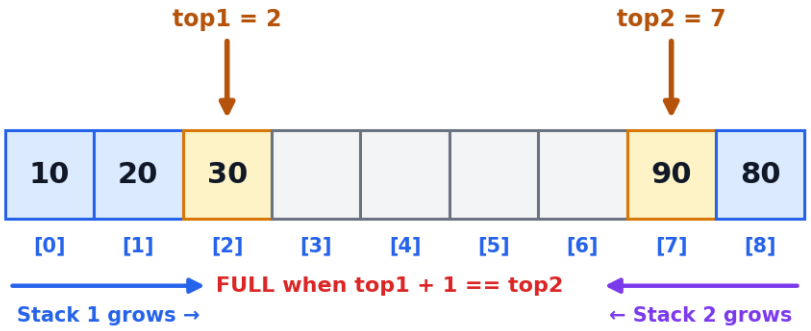


Fig: Two stacks sharing a single array – no space is wasted

Advantages	Disadvantages
Very simple to code	Fixed size – overflow possible
Fast – direct indexing, no pointers	Memory wasted if the stack stays small
Less memory per element	Size cannot grow at run time

3.4 Stack using a Linked List

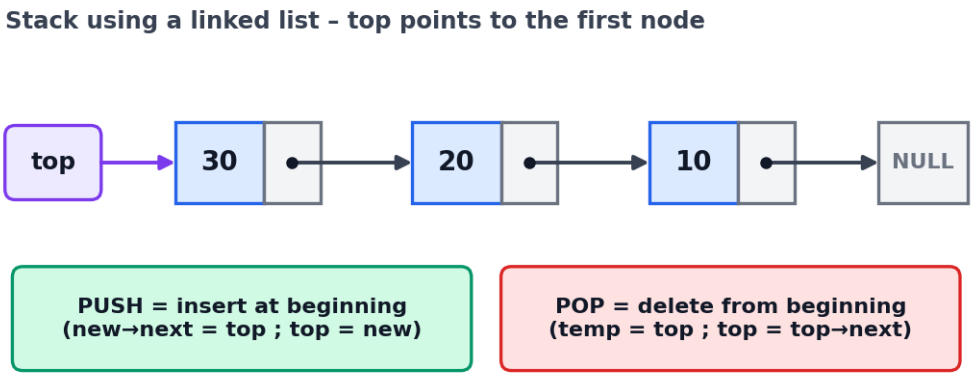


Fig: Linked-list stack – the head of the list acts as the top

- Each element is a **node**; **top** points to the first node.
- **Push = insert at the beginning**, **Pop = delete from the beginning** – both O(1).
- No overflow (until system memory is exhausted); stack is **empty when top == NULL**.

```
struct Node { int data; struct Node *next; };
struct Node *top = NULL;                        // empty stack

void push(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;
    n->next = top;               // new node points to old top
    top = n;                     // new node becomes top
}
int pop() {
    if (top == NULL) { printf("Stack Underflow\n"); return -1; }
    struct Node *t = top;
    int x = t->data;
    top = top->next;             // move top down
    free(t);
    return x;
}
int peek() { return (top == NULL) ? -1 : top->data; }
```

Basis	Array stack	Linked-list stack
Size	Fixed (MAX)	Dynamic
Overflow	Possible	Almost never
Memory	Contiguous, no pointers	Extra pointer per node
Speed	Slightly faster	Slightly slower (malloc / free)
Use when	Maximum size is known	Size is unpredictable

3.5 Applications of Stacks

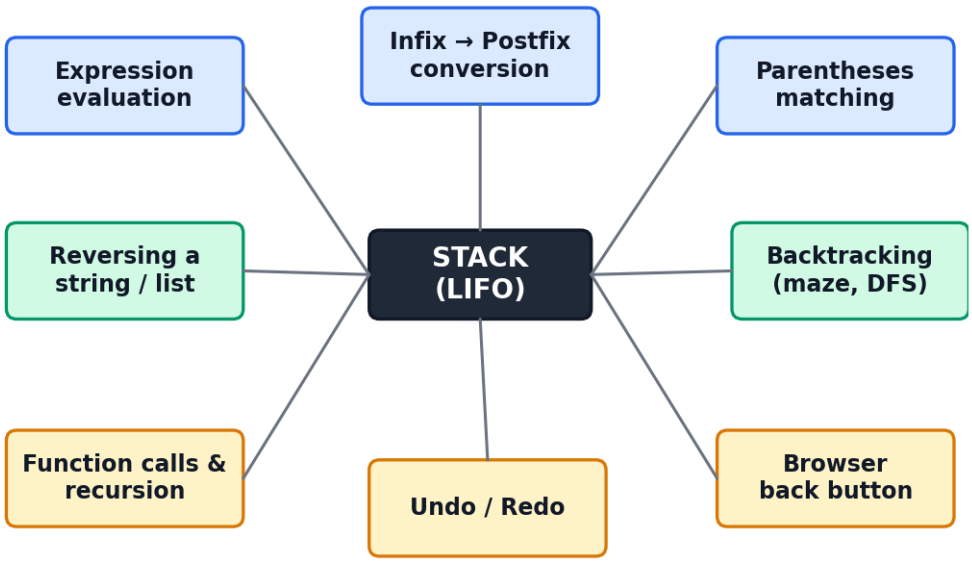


Fig: Where stacks are used

A. Expression evaluation

An expression can be written in three notations. **Computers prefer postfix** because it needs **no brackets** and no precedence rules – it can be evaluated with a single left-to-right scan using a stack.

Notation	Operator position	Example (A + B)	Example: (A + B) * C
Infix	Between operands	A + B	(A + B) * C
Prefix (Polish)	Before operands	+ A B	* + A B C
Postfix (Reverse Polish)	After operands	A B +	A B + C *

Operator	Precedence	Associativity
^ (power)	Highest (3)	Right → Left
*** / %**	2	Left → Right
+ −	Lowest (1)	Left → Right

Infix → Postfix conversion (operator stack)

1. Scan the expression from **left to right**.
2. **Operand** → send directly to output.
3. **“(”** → push on stack.
4. **“)”** → pop to output until “(” is found; discard both brackets.
5. **Operator** → while the stack top has **higher or equal precedence**, pop it to output; then push the current operator.
6. At the end, pop everything left on the stack to output.

Example 1: A + B * C − D → **A B C * + D −**

Symbol	Stack (bottom → top)	Output	Action
A	—	A	operand
+	+	A	push
B	+	A B	operand
*****	+ *	A B	* > + → push
C	+ *	A B C	operand
−	−	A B C * +	pop * and + (≥ −), push −
D	−	A B C * + D	operand
end	—	A B C * + D −	pop all

Example 2: (A + B) * C → **A B + C ***

Symbol	Stack	Output
(	(	—
A	(	A
+	(+	A
B	(+	A B
)	— (pop + until “(”)	A B +
*****	*	A B +
C	*	A B + C
end	—	A B + C *

```
int prec(char c) { return c=='^' ? 3 : (c=='*'||c=='/') ? 2 : (c=='+'||c=='-') ? 1 : 0; }

void infixToPostfix(char in[], char out[]) {
    int k = 0;
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (isalnum(c))      out[k++] = c;                          // operand
        else if (c == '(')   push(c);
        else if (c == ')') { while (peek() != '(') out[k++] = pop(); pop(); }
        else { while (!isEmpty() && prec(peek()) >= prec(c)) out[k++] = pop();
               push(c); }                                          // operator
    }
    while (!isEmpty()) out[k++] = pop();
    out[k] = '\0';
}
```

Infix → Prefix conversion

1. **Reverse** the infix expression.
2. Swap every **“(” with “)”** and vice versa.
3. Convert the result to **postfix** (use the same algorithm).
4. **Reverse** the postfix result – this is the prefix.

Example: A + B * C → reverse: C * B + A → postfix: C B * A + → reverse: **+ A * B C**

Evaluating a postfix expression

1. Scan left to right.
2. **Operand** → push.
3. **Operator** → pop **two** operands: first pop = **b** (right), second pop = **a** (left); compute **a op b**; push the result.
4. At the end the single value on the stack is the answer.

Example: 5 6 2 + * 12 4 / − = 5 × (6 + 2) − 12 ÷ 4 = **37**

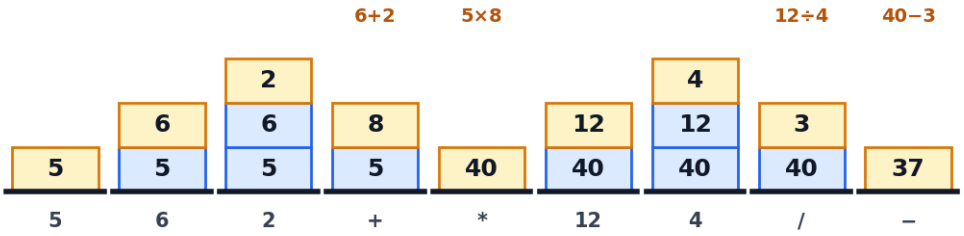


Fig: Stack contents after each symbol of the postfix expression

```
int evalPostfix(char e[]) {
    for (int i = 0; e[i]; i++) {
        if (isdigit(e[i])) push(e[i] - '0');                  // operand
        else {
            int b = pop(), a = pop();                         // note the order: b first
            switch (e[i]) { case '+': push(a + b); break;  case '-': push(a - b); break;
                            case '*': push(a * b); break;  case '/': push(a / b); break; }
        }
    }
    return pop();
}
```

More practice (solved):

Infix	Postfix	Prefix
A + B * C	A B C * +	+ A * B C
(A + B) * (C − D)	A B + C D − *	* + A B − C D

Infix	Postfix	Prefix
A * B + C / D	A B * C D / +	+ * A B / C D
A − B − C	A B − C − (left to right)	− − A B C

COMMON MISTAKE
In subtraction and division the **order matters**: result = (second popped) **op** (first popped). Writing b − a instead of a − b is the most common error.

B. Parenthesis matching (balanced brackets)

- Scan the string; **push every opening** bracket ([{.
- On a **closing** bracket, pop and check that it is the **matching** opening bracket (stack empty → unbalanced).
- At the end the stack must be **empty**.

Expression	Process	Result
{ [( )] }	push { [(→) matches (→] matches [→ } matches { → stack empty	**Balanced**
([ )]	push ([→) meets [on top → mismatch	**Not balanced**
(( )	push ((→) matches → one “(” left	**Not balanced**

```
int isBalanced(char s[]) {
    for (int i = 0; s[i]; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') push(c);                  // opening
        else if (c == ')' || c == ']' || c == '}') {
            if (isEmpty()) return 0;                                    // nothing to match
            char o = pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') ||
                (c == '}' && o != '{')) return 0;
        }
    }
    return isEmpty();                                       // must be empty at end
}
```

C. Reversing a string / list

Push every character, then pop them one by one. Because of LIFO, the order comes out **reversed**. Same idea reverses a linked list or an array.

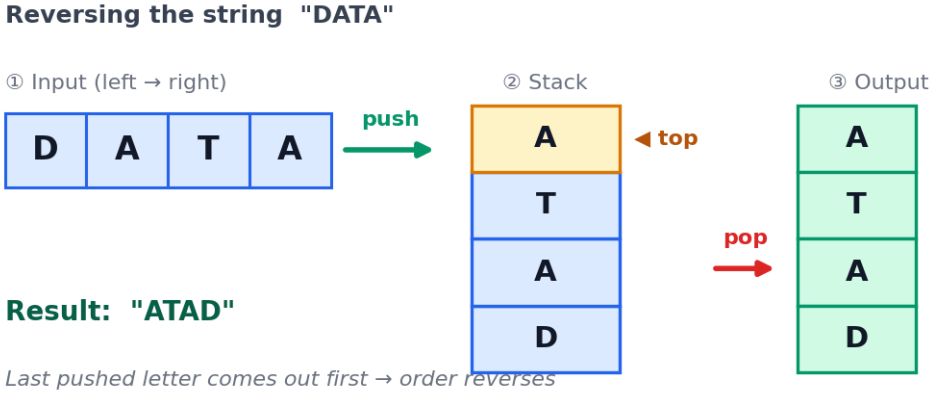


Fig: Reversing “DATA” with a stack

```
void reverse(char s[]) {
    for (int i = 0; s[i]; i++) push(s[i]);          // push all characters
    for (int i = 0; s[i]; i++) s[i] = pop();        // pop them back -> reversed
}
```

- **Palindrome check:** reverse with a stack and compare with the original.

D. Backtracking

Backtracking = try a choice; if it leads to a dead end, **undo it (pop)** and try the next choice. The stack remembers the path taken so far.

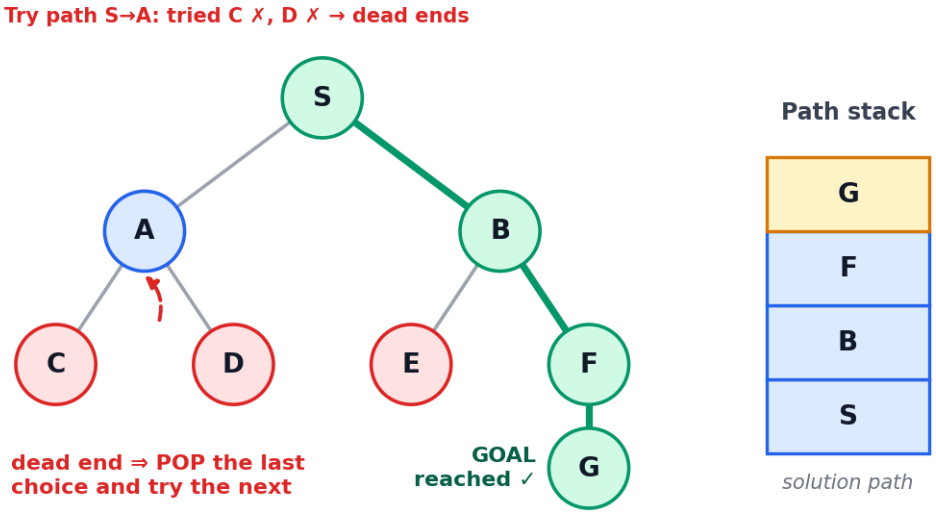


Fig: Backtracking in a maze / decision tree – dead ends are popped from the path stack

- **Push** a position when you move forward.
- **Pop** when there is no unvisited neighbour (dead end).
- When the goal is reached, the stack holds the **solution path**.
- Used in: **maze solving, N-Queens, Sudoku, depth-first search (DFS)** in graphs.

E. Function calls and recursion

- Every function call pushes an **activation record (stack frame)** – parameters, local variables and return address.
- When the function returns, its frame is **popped** and control goes back to the caller.
- Deep or infinite recursion → **stack overflow** error.

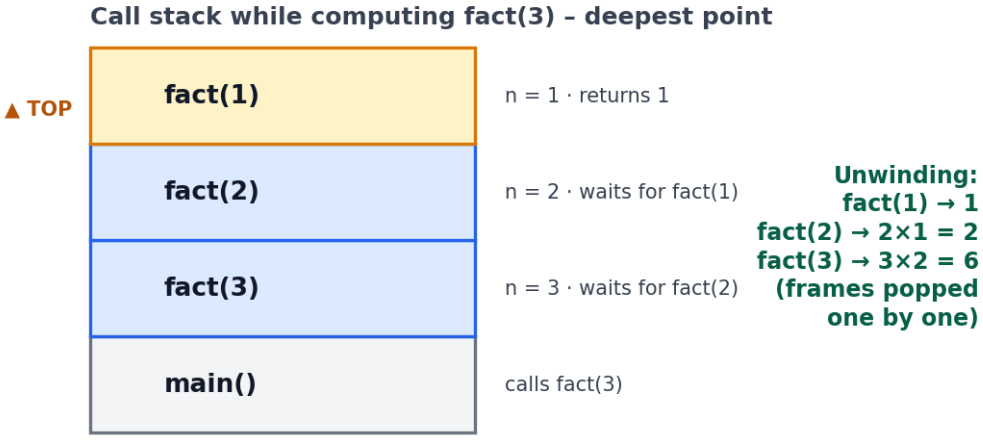


Fig: System call stack for fact(3)

F. Decimal to binary conversion

Divide the number by 2 repeatedly and **push each remainder**. The **last remainder is the most significant bit**, so popping gives the binary number in the correct order.

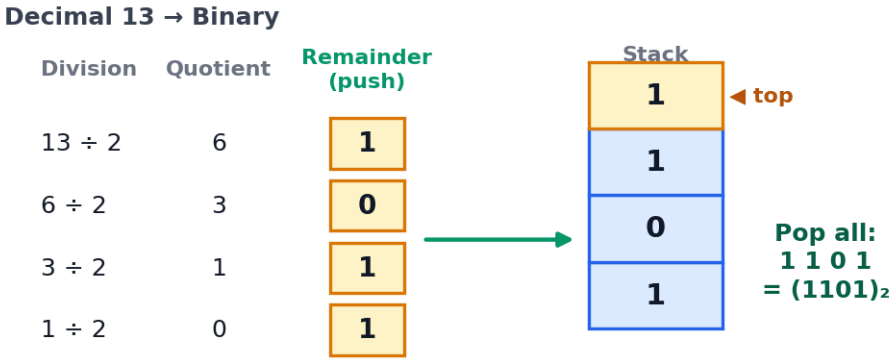


Fig: Converting 13 to binary using a stack

```
void toBinary(int n) {
    while (n > 0) { push(n % 2); n = n / 2; }     // push remainders
    while (!isEmpty()) printf("%d", pop());       // pop -> correct order
}
```

G. Other applications

- **Undo / Redo** – two stacks of actions.
- **Browser history** – Back button pops the last page.
- **Converting decimal to binary** – push remainders, pop to print.
- **Syntax checking** by compilers (brackets, begin–end).
- **Memory management** – local variables are stored on the stack.

3.6 Quick Revision & Practice

ONE-PAGE SUMMARY

- Stack = **LIFO**; all operations at the **TOP**; push, pop, peek are **O(1)**.
- Array stack: top = −1 empty, top = MAX−1 full. Linked stack: top = NULL empty.
- Postfix needs **no brackets**; evaluate with an operand stack; convert with an operator stack.
- Reversal, bracket matching, backtracking and recursion all rely on LIFO.
- Overflow = push on full. Underflow = pop on empty.

Likely exam questions

2 marks

- Define stack. Why is it called LIFO?
- What are overflow and underflow?
- Convert (A+B)*C to postfix.
- List any four applications of stacks.

5 marks

- Write algorithms for PUSH and POP using an array.
- Compare stack implementation using array and linked list.
- Explain how a stack is used to check balanced parentheses.
- Evaluate the postfix expression 2 3 1 * + 9 − using a stack.

10 marks

- Explain infix-to-postfix conversion with algorithm, example and program.
- Explain stack ADT with array and linked-list implementations and applications.

Try it yourself

1. Convert **A * (B + C) / D** to postfix. *(Answer: A B C + * D /)*
2. Evaluate **8 2 3 ^ / 2 3 * + 5 −** where ^ is power. *(Answer: 8 ÷ 8 + 6 − 5 = 2)*
3. Is **{ ([ ]) })** balanced? Show the stack.
4. Write a program to convert a decimal number to binary using a stack.

DATA STRUCTURES · DIGITAL NOTES

UNIT IV

Queues and Deques

FIFO structure · Array & linked queues · Circular queue · BFS · Scheduling · Deques

Roadmap of this unit

4.1	Introduction to queues: properties & operations
4.2	Implementing queues using arrays (linear & circular)
4.3	Implementing queues using linked lists
4.4	Priority queue & types of queues
4.5	Applications: BFS, scheduling and more
4.6	Deques: operations, types and applications

4.1 Introduction to Queues

DEFINITION

Queue = a linear data structure in which elements are **inserted at one end (REAR)** and **deleted from the other end (FRONT)**. It follows **FIFO** – **F**irst **I**n, **F**irst **O**ut.

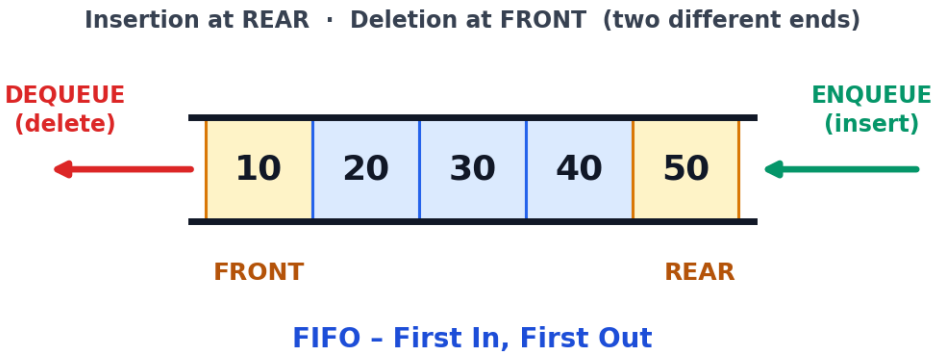


Fig: Queue – enqueue at the rear, dequeue at the front

Real-life examples

- People standing in a **ticket / billing counter line**.
- **Print jobs** waiting for a printer.
- Vehicles at a **toll gate**.
- **Customer-care calls** held in order of arrival.

Properties

- **FIFO** – the earliest arrival is served first (fairness).
- Two pointers: **front** (where we delete) and **rear** (where we insert).
- No random access – only the front element can be removed.
- Every basic operation is **O(1)**.

Operations

Operation	Meaning	Time
enqueue(x)	Insert x at the **rear** (check overflow)	O(1)
dequeue()	Remove and return the **front** element (check underflow)	O(1)
front() / peek()	Read the front element without removing	O(1)
isEmpty()	True if queue has no element	O(1)
isFull()	True if no space (array queue)	O(1)

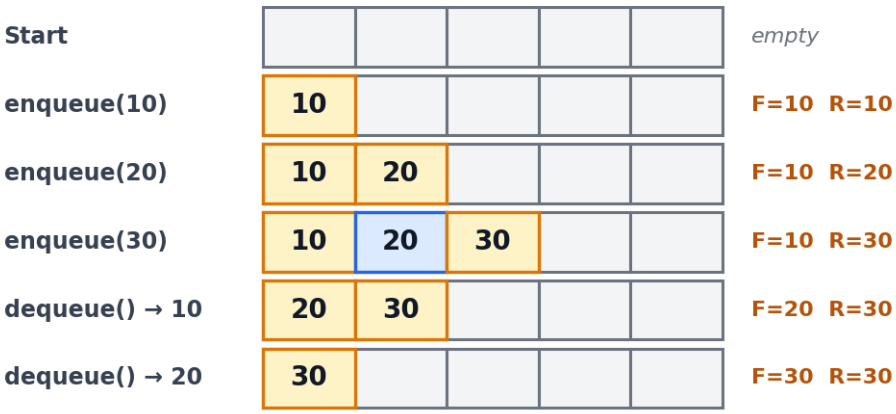


Fig: Queue state after each operation – F = front value, R = rear value

Stack vs Queue

Basis	Stack	Queue
Principle	LIFO	FIFO
Ends used	One end (top)	Two ends (front, rear)
Insert / Delete	push / pop at top	enqueue at rear / dequeue at front
Pointers	top	front and rear
Examples	Undo, recursion, expression evaluation	Scheduling, BFS, buffering

4.2 Queue using an Array

A. Linear (simple) queue

- Array **queue[MAX]**, with **front = −1** and **rear = −1** (empty).
- enqueue: if rear == MAX−1 → overflow; else **rear++**, store (and set front = 0 for the first element).
- dequeue: if front == −1 or front > rear → underflow; else take queue[front] and **front++**.

```
#define MAX 5
int queue[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;            // first element
    queue[++rear] = x;
}
int dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return -1; }
    return queue[front++];
}
```

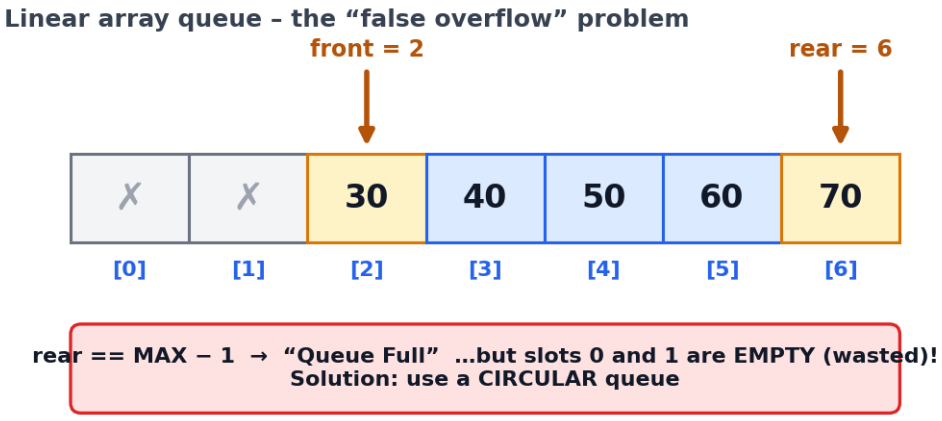


Fig: Problem of the linear queue – freed slots at the front are never reused

B. Circular queue

The array is treated as a **ring**: after the last index the pointers wrap around to index 0. Freed slots are reused, so there is **no false overflow**.

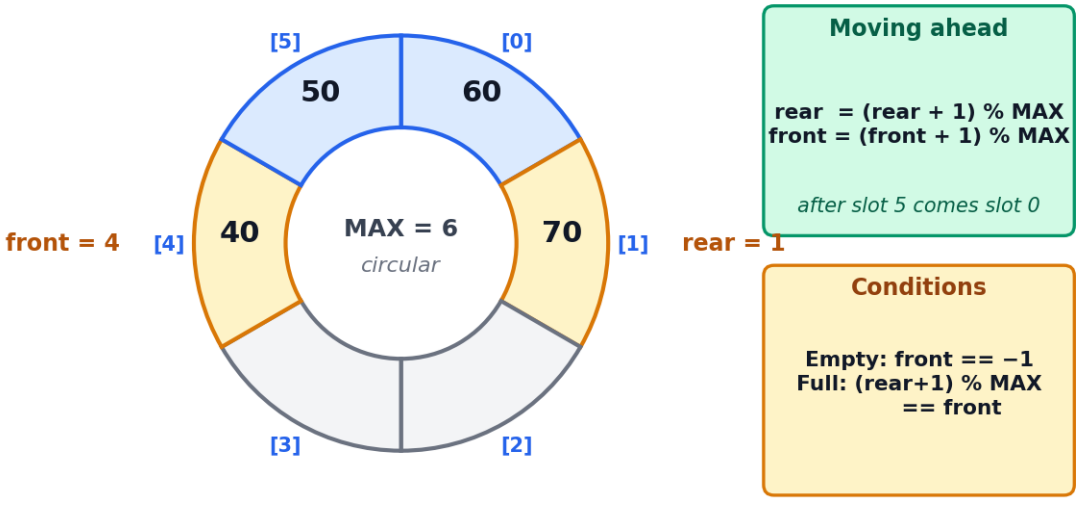


Fig: Circular queue with MAX = 6 – elements 40, 50, 60, 70 wrap around the end of the array

```
#define MAX 5
int cq[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front) { printf("Queue Full\n"); return; }
    if (front == -1) front = 0;                 // first insertion
    rear = (rear + 1) % MAX;                    // wrap around
    cq[rear] = x;
}
int dequeue() {
    if (front == -1) { printf("Queue Empty\n"); return -1; }
    int x = cq[front];
    if (front == rear) front = rear = -1;       // last element removed
    else front = (front + 1) % MAX;
    return x;
}
```

A circular queue of size MAX can hold at most MAX elements with this test

Dry run of a circular queue (MAX = 4)

Operation	front	rear	cq[0] [1] [2] [3]	Remark
(start)	−1	−1	_ _ _ _	empty
enqueue(10)	0	0	10 _ _ _	first element
enqueue(20), (30)	0	2	10 20 30 _	
dequeue()	1	2	(10) 20 30 _	returns 10
enqueue(40)	1	3	(10) 20 30 40	rear at last index
enqueue(50)	1	0	50 20 30 40	**wraps to index 0**; now full
enqueue(60)	1	0	50 20 30 40	(0+1)%4 == 1 → **Full**

Basis	Linear queue	Circular queue
Pointer movement	rear++ / front++	(rear+1) % MAX
Memory reuse	No – wasted slots	**Yes**
Full condition	rear == MAX − 1	(rear + 1) % MAX == front
Empty condition	front == −1 or front > rear	front == −1
Efficiency	Poor	Better

EXAM TIP
Remember the formula **(index + 1) % MAX** – it is the heart of every circular-queue question. Dry-run with MAX = 4 to see how the wrap-around works.

4.3 Queue using a Linked List

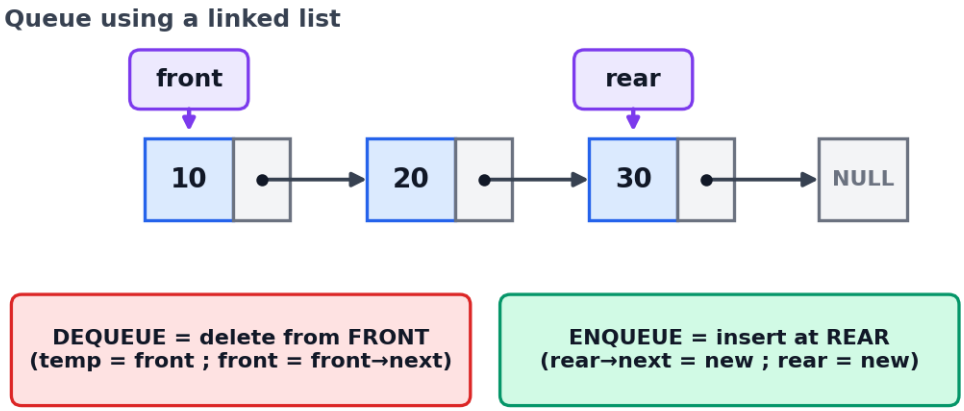


Fig: Linked queue – front and rear pointers

- Each element is a node; **front** points to the first node, **rear** to the last.
- **Enqueue** = insert at the end (O(1) thanks to rear). **Dequeue** = delete the first node (O(1)).
- Dynamic size – no overflow; empty when **front == NULL**.

```
struct Node { int data; struct Node *next; };
struct Node *front = NULL, *rear = NULL;

void enqueue(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;  n->next = NULL;
    if (rear == NULL) front = rear = n;     // first node
    else { rear->next = n; rear = n; }      // attach at the end
}
int dequeue() {
    if (front == NULL) { printf("Queue Underflow\n"); return -1; }
    struct Node *t = front;  int x = t->data;
    front = front->next;
    if (front == NULL) rear = NULL;         // queue became empty
    free(t);
    return x;
}
```

Basis	Array queue	Linked queue
Size	Fixed	Dynamic
Overflow	Possible	Practically none
Extra memory	None	One pointer per node
Implementation	Simple, needs circular logic	Pointer handling, free()

COMMON MISTAKE
When the last node is dequeued, set **rear = NULL** as well – otherwise rear keeps pointing to freed memory.

4.4 Priority Queue and Types of Queues

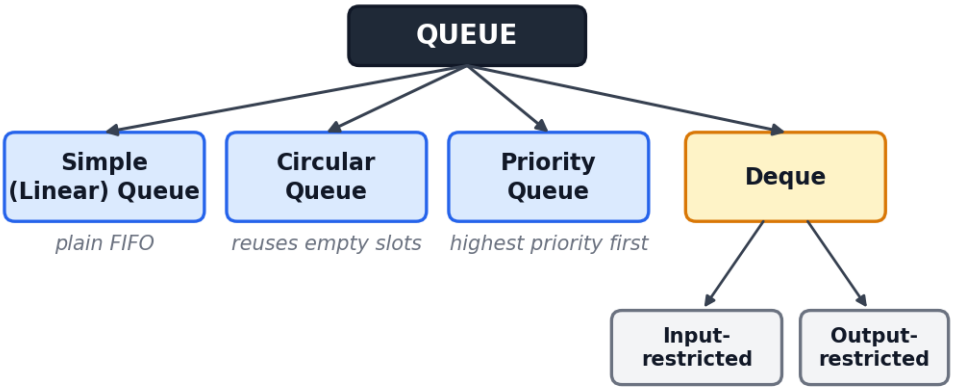


Fig: Family of queues

Type	Rule	Use
Simple queue	FIFO, linear array	Basic waiting lines
Circular queue	FIFO with wrap-around	Buffers, CPU scheduling
Priority queue	Element with **highest priority** leaves first; equal priority → FIFO	OS job scheduling, hospital emergency, Dijkstra
Deque	Insert / delete at both ends	Sliding window, palindromes

Priority queue – key points

- Each element carries a **priority** value; **dequeue** removes the highest (or lowest) priority element, not the oldest.
- Implementation: **unsorted array / list** (insert O(1), delete O(n)), **sorted list** (insert O(n), delete O(1)) or a **heap** (both O(log n)).

Arrival order	Task (priority)	Dequeue order
1	Print report (3)	3rd
2	Fire alarm (1 = highest)	**1st**
3	Backup (4)	4th
4	Email sync (2)	2nd

Priority queue using a sorted linked list

```
struct PNode { int data, pri; struct PNode *next; };
struct PNode *head = NULL;                        // highest priority (smallest number) first

void insertP(int x, int p) {
    struct PNode *n = (struct PNode*) malloc(sizeof(struct PNode));
    n->data = x;  n->pri = p;
    if (head == NULL || p < head->pri) { n->next = head; head = n; return; }
    struct PNode *t = head;
    while (t->next != NULL && t->next->pri <= p) t = t->next;   // FIFO if equal
    n->next = t->next;  t->next = n;
}
int deleteP() {                                   // remove the highest priority element
    struct PNode *t = head;  int x = t->data;
    head = head->next;  free(t);  return x;
}
```

Insertion O(n), deletion O(1)

Queue and stack from each other

Task	Idea
Queue using two stacks	enqueue → push on stack S1. dequeue → if S2 is empty, pop everything from S1 and push into S2 (reverses the order); then pop S2.
Stack using two queues	push → enqueue into Q2, then move all of Q1 behind it; swap names. pop → dequeue from Q1.
Reverse a queue	Dequeue every element into a stack, then pop and enqueue them back.

4.5 Applications of Queues

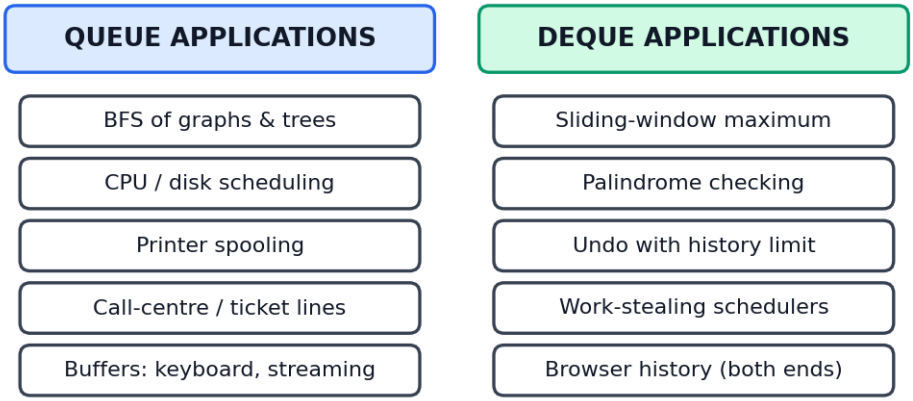


Fig: Typical applications of queues and deques

A. Breadth-First Search (BFS)

- BFS visits a graph **level by level**: start vertex, then all its neighbours, then their neighbours...
- A **queue** keeps the vertices waiting to be explored (FIFO ensures the nearest vertices are served first).
- Uses: **shortest path in an unweighted graph**, social-network "friends of friends", web crawling, level-order traversal of trees.

Algorithm

1. Mark the start vertex visited and **enqueue** it.
2. While the queue is not empty: **dequeue** a vertex v and visit (print) it.
3. For every unvisited neighbour w of v: mark visited and **enqueue** w.

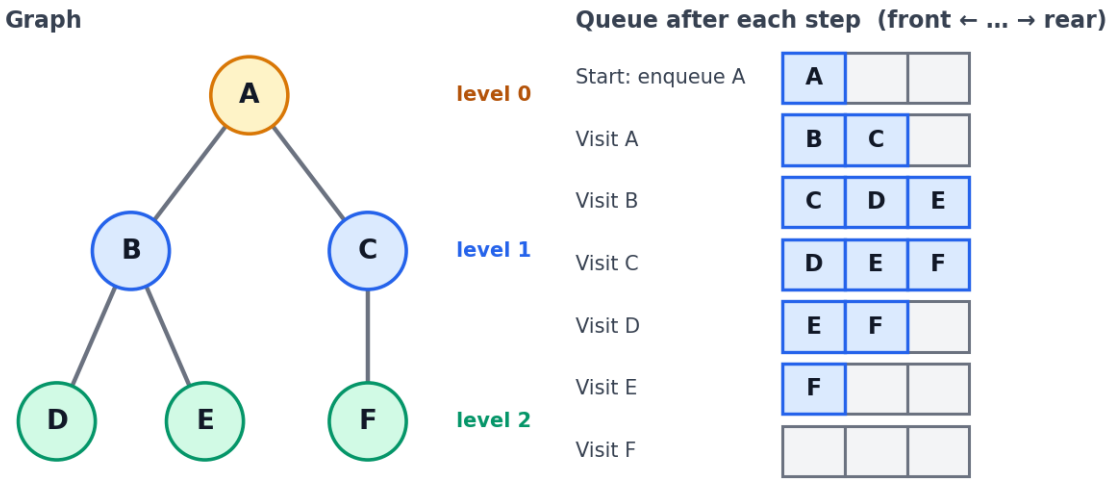


Fig: BFS from A – order of visiting: A → B → C → D → E → F

```
void BFS(int g[N][N], int start) {                // adjacency matrix g
    int visited[N] = {0};
    enqueue(start);  visited[start] = 1;
    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);
        for (int w = 0; w < N; w++)
            if (g[v][w] && !visited[w]) { visited[w] = 1; enqueue(w); }
    }
}
```

Time O(V + E) with adjacency lists, O(V²) with an adjacency matrix

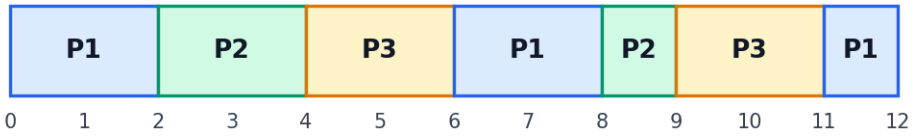
REMEMBER

BFS → **Queue** (level by level). DFS → **Stack** (go deep, then backtrack).

B. CPU scheduling (round robin)

- Ready processes wait in a **circular queue**.
- The CPU gives each process a fixed **time quantum**; if unfinished, it goes to the **rear** of the queue.
- First-Come-First-Served (FCFS) scheduling is a plain FIFO queue.

Round-robin scheduling, time quantum = 2 (P1 = 5, P2 = 3, P3 = 4 units)



Time units →

The ready queue rotates: P1 → P2 → P3 → P1 → … (finished processes leave the queue)

Fig: Gantt chart of round-robin scheduling

Process	Burst time	Finish time	Turnaround (finish − arrival 0)	Waiting (turnaround − burst)
P1	5	12	12	7
P2	3	9	9	6
P3	4	11	11	7

Ready-queue snapshots for the chart above (front on the left):

Time	CPU runs	Ready queue after the slice
0 – 2	P1 (3 left)	P2, P3, P1
2 – 4	P2 (1 left)	P3, P1, P2
4 – 6	P3 (2 left)	P1, P2, P3
6 – 8	P1 (1 left)	P2, P3, P1
8 – 9	P2 (done)	P3, P1
9 – 11	P3 (done)	P1
11 – 12	P1 (done)	empty

C. Other applications

- **Printer spooler** and **disk scheduling** – requests served in arrival order.
- **Buffers** – keyboard input, video streaming, data between fast and slow devices.
- **Call centres, ticket booking** – customers served in order.
- **Level-order traversal** of trees (Unit V).
- **Simulation** of real-world waiting lines (banks, traffic).

4.6 Deques (Double-Ended Queues)

DEFINITION
Deque (pronounced “deck”) = a linear structure that allows **insertion and deletion at both ends** – front and rear.

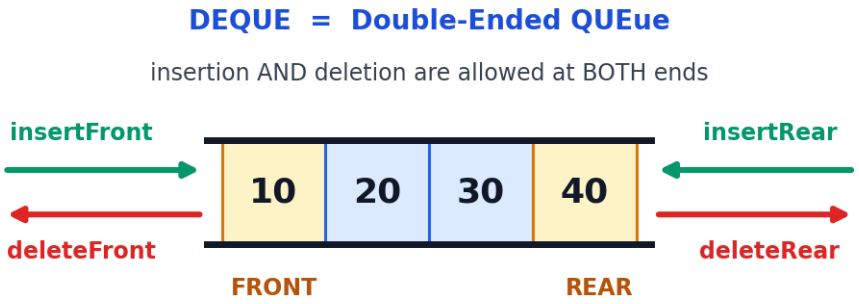


Fig: A deque supports four basic operations

Operation	Action	Time
insertFront(x)	Add at the front	O(1)
insertRear(x)	Add at the rear	O(1)
deleteFront()	Remove from the front	O(1)
deleteRear()	Remove from the rear	O(1)
getFront() / getRear()	Read either end	O(1)

Types of deque

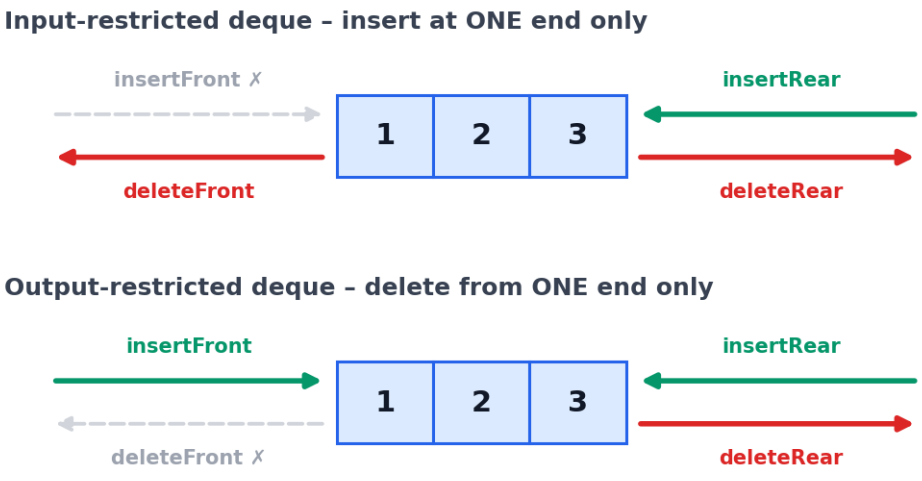


Fig: Input-restricted and output-restricted deques

- **Input-restricted:** insertion at **one end only**, deletion at **both** ends.
- **Output-restricted:** deletion at **one end only**, insertion at **both** ends.
- A deque used from **one end only** behaves like a **stack**; used with insert-rear / delete-front it behaves like a **queue**.

Implementation

- **Circular array:** front and rear wrap around; insertFront uses **front = (front − 1 + MAX) % MAX**.
- **Doubly linked list:** prev and next pointers make all four operations O(1) with no size limit.

```
int dq[MAX], front = -1, rear = -1;                    // circular array deque

void insertFront(int x) {
    if ((rear + 1) % MAX == front) { printf("Full\n"); return; }
    if (front == -1) front = rear = 0;
    else front = (front - 1 + MAX) % MAX;              // move backwards with wrap
    dq[front] = x;
}
void insertRear(int x) {
    if ((rear + 1) % MAX == front) { printf("Full\n"); return; }
    if (front == -1) front = rear = 0;
    else rear = (rear + 1) % MAX;
    dq[rear] = x;
}
int deleteFront() {
    if (front == -1) { printf("Empty\n"); return -1; }
    int x = dq[front];
    if (front == rear) front = rear = -1; else front = (front + 1) % MAX;
    return x;
}
int deleteRear() {
    if (front == -1) { printf("Empty\n"); return -1; }
    int x = dq[rear];
    if (front == rear) front = rear = -1; else rear = (rear - 1 + MAX) % MAX;
    return x;
}
```

Palindrome check using a deque (example: “MADAM”)

Step	Deque	Compare front and rear	Result
1	M A D A M	M = M	delete both ends
2	A D A	A = A	delete both ends
3	D	only one element left	**Palindrome**

Applications of deques

- **Palindrome check** – compare the front and rear characters, delete both, repeat.
- **Sliding-window maximum / minimum** problems.
- **Undo / redo with limited history** – oldest action is dropped from one end, newest added at the other.
- **Work stealing** in multi-core schedulers; **A-Steal** task queues.
- Browser history and **both-direction** traversal of items.

4.7 Quick Revision & Practice

ONE-PAGE SUMMARY

- Queue = **FIFO**; insert at **rear**, delete at **front**; all operations O(1).
- Linear array queue wastes freed slots → use **circular queue** with **(i+1) % MAX**.
- Linked queue: front for delete, rear for insert; empty when front == NULL.
- Priority queue: highest priority leaves first.
- BFS uses a **queue**; round-robin scheduling uses a **circular queue**.
- Deque = both ends; input-restricted / output-restricted variants.

Likely exam questions

2 marks

- Define queue. State its principle.
- Differentiate stack and queue.
- What is a circular queue? Why is it needed?
- What is a deque? Name its two types.

5 marks

- Write algorithms for enqueue and dequeue in a circular queue.
- Explain queue implementation using a linked list.
- Explain BFS with an example and trace the queue.
- Explain priority queue with an example.

10 marks

- Explain the queue ADT with array and linked-list implementations and applications.
- Explain deque operations, types and implementation with diagrams.

Try it yourself

1. In a circular queue of size 5, perform: enq 10, 20, 30, 40; deq, deq; enq 50, 60, 70. Show front and rear after each step.
2. Run BFS on a graph with edges 1-2, 1-3, 2-4, 3-4, 4-5 from vertex 1. *(Answer: 1 2 3 4 5)*
3. Perform round robin (quantum 3) for P1 = 7, P2 = 4, P3 = 5 and draw the Gantt chart.
4. Reverse the first k elements of a queue using a stack.

DATA STRUCTURES · DIGITAL NOTES

UNIT V

Trees and Hashing

Trees · Binary Search Tree (insert, delete, traverse) · Hash functions · Collision handling · Hash tables

Roadmap of this unit

5.1	Introduction to trees & terminology
5.2	Binary trees and Binary Search Tree (BST)
5.3	BST insertion, deletion and traversal
5.4	Hashing: hash functions & hash tables
5.5	Collision resolution: chaining & open addressing
5.6	Applications of hashing: unique IDs, caching

5.1 Introduction to Trees

DEFINITION

Tree = a **non-linear, hierarchical** data structure made of **nodes** connected by **edges**, with one special node called the **root**. A tree with n nodes has exactly **n − 1 edges** and **no cycles**.

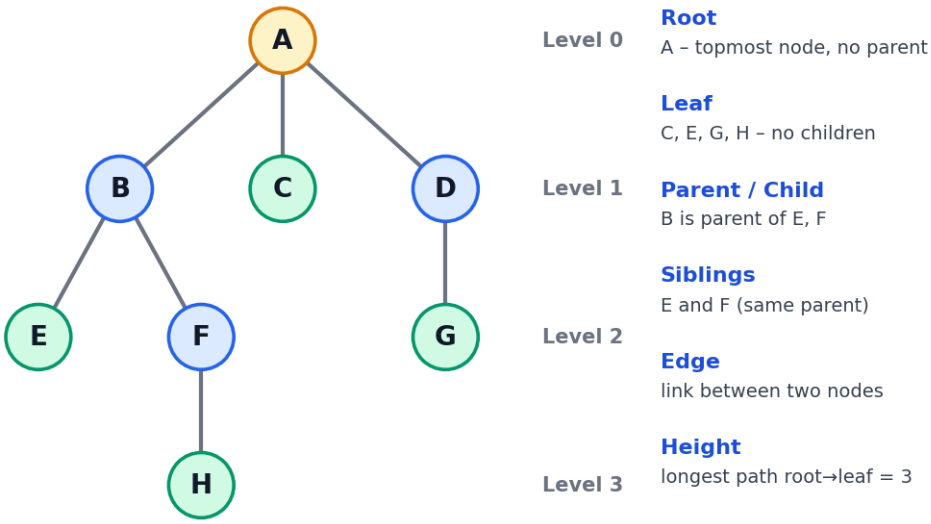


Fig: A general tree with its terminology

Term	Meaning
Root	Top node – the only node without a parent
Parent / Child	Node directly above / below another node
Siblings	Nodes with the same parent
Leaf (external node)	Node with no children
Internal node	Node with at least one child
Degree of a node	Number of children of that node

Term	Meaning
Level	Distance from the root (root = level 0)
Height of tree	Number of edges on the longest path from root to a leaf
Subtree	A node together with all its descendants
Ancestor / Descendant	Any node on the path up to the root / any node below

Why trees?

- Natural for **hierarchical data**: file systems, organisation charts, HTML DOM.
- Faster **search / insert / delete** than lists – O(log n) in a balanced tree.
- Basis of expression trees, heaps, tries, decision trees, database indexes.

5.2 Binary Trees and Binary Search Tree

DEFINITION
Binary tree = a tree in which every node has **at most two children**, called the **left child** and the **right child**.

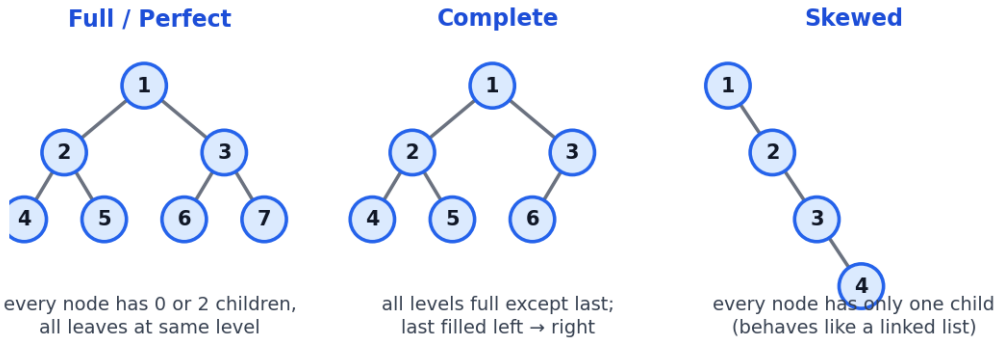


Fig: Common shapes of binary trees

- Maximum nodes at level L = 2^L.
- Maximum nodes in a tree of height h = $2^{h+1} - 1$.
- A binary tree with n nodes has height between $\log_2 n$ (balanced) and $n - 1$ (skewed).

```
struct Node {
    int data;
    struct Node *left, *right;          // two child pointers
};
struct Node* newNode(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;  n->left = n->right = NULL;
    return n;
}
```

Node of a binary tree

Binary Search Tree (BST)

DEFINITION
BST = a binary tree in which, for **every node**: all values in the **left subtree are smaller** and all values in the **right subtree are larger** than the node. (Duplicates are normally not allowed.)

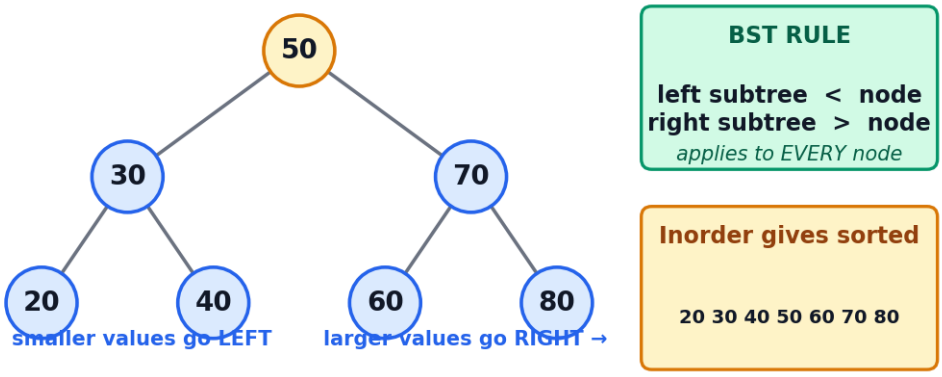


Fig: A BST – the inorder traversal always prints the keys in sorted order

Operation	Balanced BST	Skewed BST (worst)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Space	O(n)	O(n)

REMEMBER
Inserting already-sorted keys (10, 20, 30, 40…) creates a **skewed tree** that behaves like a linked list – this is why balanced trees (AVL, Red-Black) exist.

5.3 BST Operations

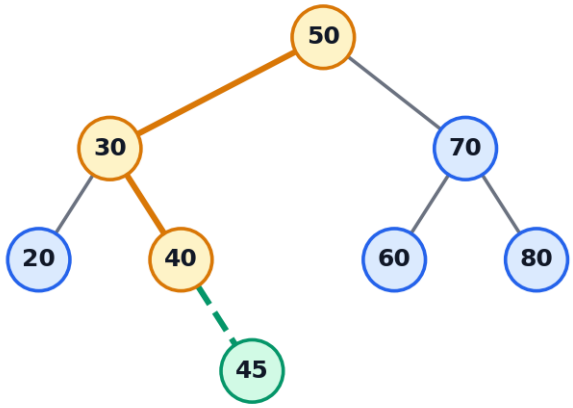
A. Searching

1. Start at the root.
2. If key == node → **found**.
3. If key < node → go **left**; if key > node → go **right**.
4. If you reach NULL → **not found**.

```
struct Node* search(struct Node *root, int key) {
    if (root == NULL || root->data == key) return root;
    if (key < root->data) return search(root->left, key);
    return search(root->right, key);
}
```

B. Insertion

- Search for the key; the **empty position where the search ends** is where the new node goes.
- A new node is **always inserted as a leaf**.



INSERT 45

① 45 < 50 → go LEFT

② 45 > 30 → go RIGHT

③ 45 > 40 → go RIGHT

④ right of 40 is empty

⑤ → place 45 here (as a leaf)

Fig: Inserting 45 – follow the comparison path and attach at the empty spot

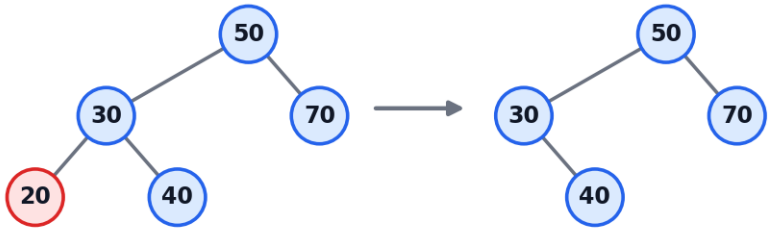
```
struct Node* insert(struct Node *root, int x) {
    if (root == NULL) return newNode(x);              // empty spot found
    if (x < root->data)      root->left  = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;                                      // duplicates ignored
}
```

Build a BST from the keys **50, 30, 70, 20, 40, 60, 80**: 50 becomes the root; 30 < 50 → left; 70 > 50 → right; 20 < 50, < 30 → left of 30; 40 → right of 30; 60 → left of 70; 80 → right of 70.

C. Deletion – three cases

Case	Node to delete has…	Action
1	**No child** (leaf)	Remove the node; set parent's pointer to NULL
2	**One child**	Replace the node by its child (parent links directly to the child)
3	**Two children**	Copy the **inorder successor** (smallest key in the right subtree) – or the inorder predecessor – into the node, then delete that successor (which has at most one child)

Case 1 – delete a LEAF node (20)



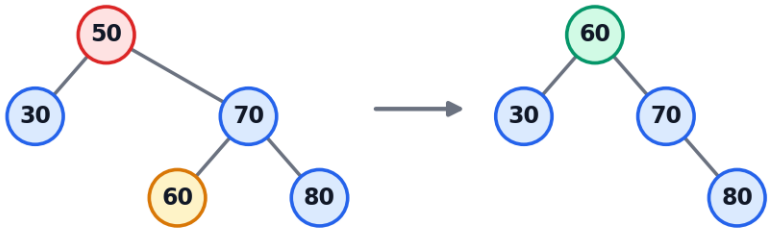
just remove it (parent's pointer = NULL)

Case 2 – delete a node with ONE child (30)



child takes the place of the deleted node

Case 3 – delete a node with TWO children (50)



replace with inorder successor 60 (smallest in right subtree), then delete 60

Fig: Deletion cases in a BST – red = node to delete, green = node that takes its place

```
struct Node* minNode(struct Node *t) { while (t->left) t = t->left; return t; }

struct Node* deleteNode(struct Node *root, int key) {
    if (root == NULL) return NULL;
    if (key < root->data)      root->left  = deleteNode(root->left, key);
    else if (key > root->data) root->right = deleteNode(root->right, key);
    else {                                              // node found
        if (root->left == NULL)  { struct Node *t = root->right; free(root); return t; }
        if (root->right == NULL) { struct Node *t = root->left;  free(root); return t; }  // 0/1 child
        struct Node *s = minNode(root->right);          // two children: inorder successor
        root->data = s->data;
        root->right = deleteNode(root->right, s->data);
    }
    return root;
}
```

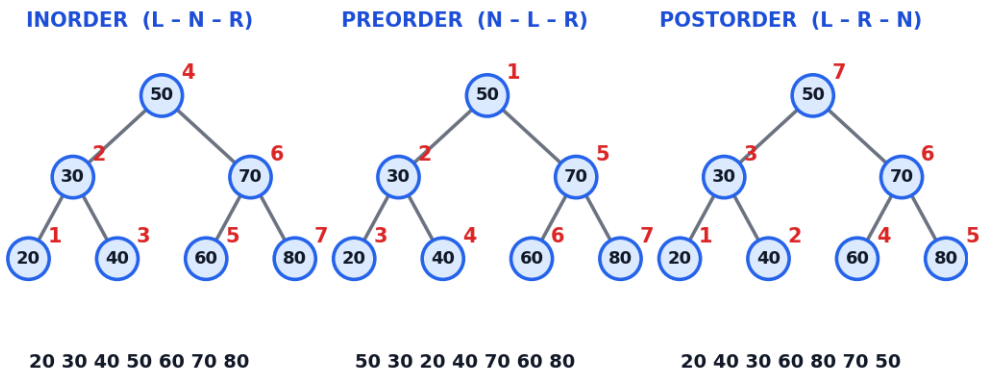
EXAM TIP

For the two-children case always write: "replace with **inorder successor** (leftmost node of the right subtree)", then show the diagram. This is a favourite 5-mark question.

5.4 Tree Traversals

Traversal = visiting every node exactly once. For a binary tree there are three depth-first orders (N = node, L = left, R = right) and one breadth-first order.

Traversal	Order	Result for the BST of Fig	Use
Inorder	L – N – R	20 30 40 50 60 70 80	Sorted output of a BST
Preorder	N – L – R	50 30 20 40 70 60 80	Copy a tree, prefix expression
Postorder	L – R – N	20 40 30 60 80 70 50	Delete a tree, postfix expression
Level order	level by level (queue)	50 30 70 20 40 60 80	BFS on a tree



red numbers = order in which nodes are visited

Fig: Traversal orders – red numbers show the visiting sequence

```
void inorder(struct Node *r) {
    if (r) { inorder(r->left);  printf("%d ", r->data);  inorder(r->right); }
}
void preorder(struct Node *r) {
    if (r) { printf("%d ", r->data);  preorder(r->left);  preorder(r->right); }
}
void postorder(struct Node *r) {
    if (r) { postorder(r->left);  postorder(r->right);  printf("%d ", r->data); }
}
```

Recursive traversals – only the position of printf changes

- **Level-order**: enqueue root; repeat – dequeue a node, print it, enqueue its left and right children (uses a **queue**, Unit IV).
- All traversals take **O(n)** time; recursive versions use O(h) stack space.
- **Memory trick:** the name tells where the **Node** is visited – **pre** = before children, **in** = between, **post** = after.

Reconstructing a tree: inorder + preorder (or inorder + postorder) together identify a unique binary tree. First element of preorder (or last of postorder) = root; elements left of the root in inorder form the left subtree.

5.5 Hashing

DEFINITION
Hashing = a technique that converts a **key** into an **array index** using a **hash function**, so that data can be stored and found in **O(1) average time**.

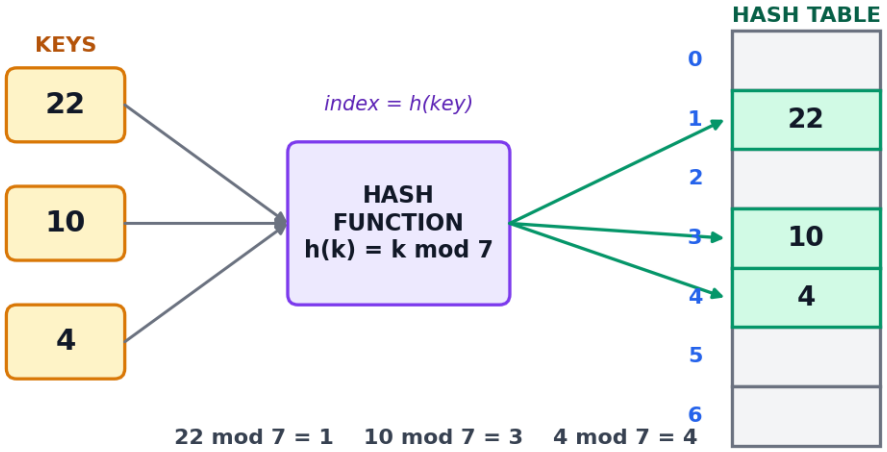


Fig: Keys are mapped to table slots by the hash function

Basic terms

Term	Meaning
Key	Value used to identify a record (roll no., username...)
Hash function h(k)	Formula that maps a key to an index 0 ... m−1
Hash table	Array of m slots (buckets) holding the records
Collision	Two different keys map to the **same** index
Load factor λ	λ = n / m (n = keys stored, m = table size) – measures how full the table is

Properties of a good hash function

- **Fast** to compute.
- **Uniform** – spreads keys evenly over the table (few collisions).
- **Deterministic** – same key always gives the same index.
- Uses **all** parts of the key.

Common hash functions

Method	Formula / idea	Example (m = 10 or 100)
Division	h(k) = k mod m (choose m as a **prime**)	1234 mod 7 = 2
Multiplication	h(k) = ⌊m · (k·A mod 1)⌋, 0 < A < 1 (A ≈ 0.618)	k = 123, m = 100 → 123 × 0.618 = 76.014 → frac 0.014 → h = 1
Mid-square	Square the key, take the middle digits	123^2 = 15129 → middle “51” → 51
Folding	Split the key into parts and add them	12345678 → 12 + 34 + 56 + 78 = 180 → 80 (last 2 digits)
String hashing	h = (Σ ascii(c_i) × 31^i) mod m	“AB” → (65×31 + 66) mod m

```
#define M 7
int hash(int key) { return key % M; }                // division method

int hashString(char *s) {                            // string hashing
    unsigned long h = 0;
    while (*s) h = h * 31 + *s++;
    return h % M;
}
```

Basic hash-table operations

Operation	Steps	Average	Worst
Insert(k)	compute h(k), place (resolve collision if any)	O(1)	O(n)
Search(k)	compute h(k), compare along the probe sequence / chain	O(1)	O(n)
Delete(k)	find the key, remove (mark deleted in open addressing)	O(1)	O(n)

5.6 Collision Resolution

A collision occurs when **$h(k_1) = h(k_2)$** for different keys. Two families of techniques handle it:

A. Separate chaining (closed addressing)

- Each slot holds a **linked list** (chain) of all keys that hash to it.
- Insert at the **head** of the list – O(1). Search = hash, then walk the list.
- Table never becomes “full”; load factor can exceed 1.
- Average search time ≈ **1 + λ**.
- Extra memory for pointers.

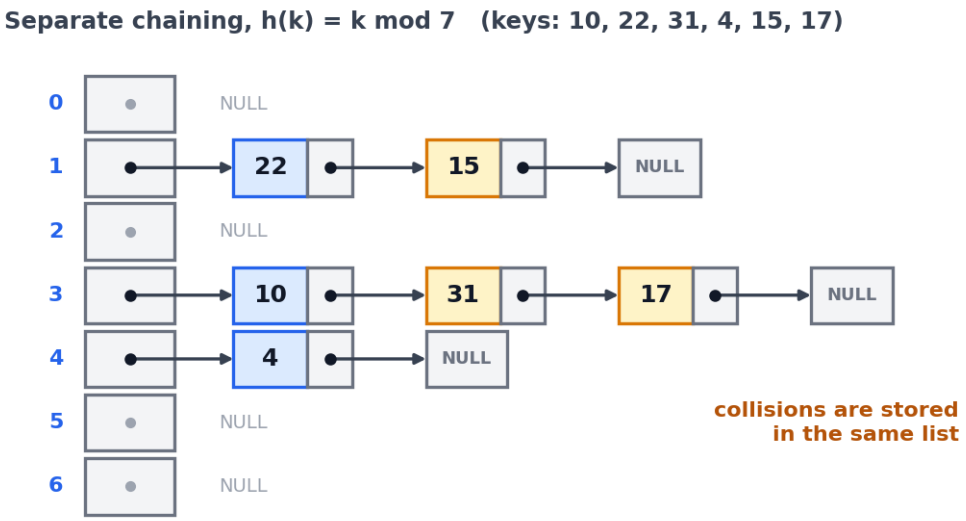


Fig: Chaining – keys 22 and 15 share slot 1; 10, 31 and 17 share slot 3

```
struct HNode { int key; struct HNode *next; };
struct HNode *table[M];                          // all NULL initially

void insertKey(int k) {
    int i = k % M;
    struct HNode *n = (struct HNode*) malloc(sizeof(struct HNode));
    n->key = k;  n->next = table[i];  table[i] = n;      // insert at head of chain
}
int searchKey(int k) {
    for (struct HNode *t = table[k % M]; t; t = t->next) if (t->key == k) return 1;
    return 0;
}
```

B. Open addressing (closed hashing)

All keys live **inside the table itself**. On collision we **probe** for another empty slot. Requires **λ ≤ 1**.

Technique	Probe sequence	Remark
Linear probing	(h(k) + i) mod m	Simple, but causes **primary clustering** (long runs of filled slots)
Quadratic probing	(h(k) + i²) mod m	Reduces primary clustering; may cause secondary clustering
Double hashing	(h₁(k) + i·h₂(k)) mod m	Best distribution; h₂(k) must never be 0, e.g. h₂ = 5 − (k mod 5)

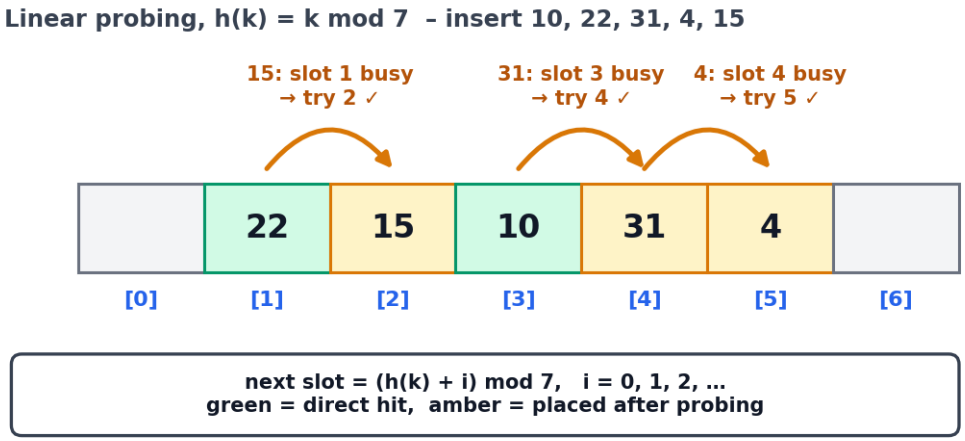


Fig: Linear probing – a colliding key moves to the next free slot

```
int table[M];                                   // -1 means empty
void insertLinear(int k) {
    int i = k % M, tries = 0;
    while (table[i] != -1 && tries++ < M) i = (i + 1) % M;   // probe next slot
    if (tries <= M) table[i] = k; else printf("Table full\n");
}
```

COMMON MISTAKE
In open addressing a deleted slot must be marked **“deleted” (tombstone)**, not simply emptied – otherwise searches for keys that probed past it will stop early and fail.

Basis	Chaining	Open addressing
Storage	Outside the table (linked lists)	Inside the table
Load factor	Can be > 1	Must be ≤ 1
Deletion	Easy	Needs tombstones
Clustering	None	Possible
Memory	Extra pointers	No pointers, better cache use

Trace (m = 7, linear probing): insert 10 → slot 3; 22 → 1; 31 → 3 busy → **4**; 4 → 4 busy → **5**; 15 → 1 busy → **2**. Final table: [_ , 22, 15, 10, 31, 4, _].

Rehashing: when λ becomes large (e.g. > 0.7) create a bigger table (about double, prime size) and re-insert every key – keeps operations near O(1).

5.7 Applications of Hashing

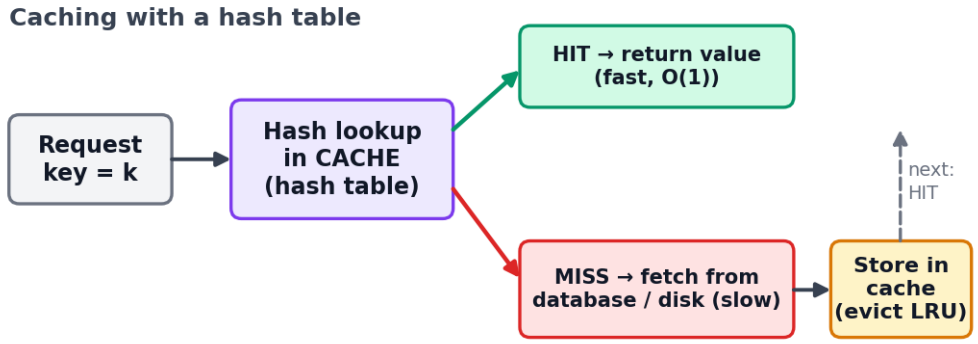


Fig: Caching – the hash table answers "is it already stored?" in O(1)

A. Unique identifier generation

- **Checksums / digests** – SHA-256 turns a file into a fixed-size fingerprint; equal fingerprints ⇒ same file (duplicate detection, integrity checks).
- **URL shorteners** – the long URL is hashed / encoded to a short unique code.
- **Version control** (Git) – every commit and file is identified by its hash.
- **Student / order IDs** – hash of name + timestamp gives an ID that is practically unique.
- **Blockchain** – each block carries the hash of the previous block.

B. Caching

- A **cache** stores recently used results so that they need not be recomputed or fetched again.
- A hash table gives **O(1) lookup** of cached entries – **hit** = found, **miss** = fetch and store.
- **LRU cache** = hash table (key → node) + doubly linked list (usage order). Both get and put are O(1); when full, the least-recently-used entry is evicted.
- Used in **browsers, CPU, DNS, databases, CDNs**.

C. Other applications

- **Dictionaries / symbol tables** in compilers and interpreters.
- **Database indexing** and joins.
- **Password storage** – only the hash of a password is stored.
- **Counting frequencies** (words, characters) and finding duplicates in O(n).
- **Sets and maps** (HashSet / HashMap, Python dict).

Structure	Search	Insert	Delete	Ordered?
Unsorted array / list	O(n)	O(1)	O(n)	No
Sorted array	O(log n)	O(n)	O(n)	Yes
BST (balanced)	O(log n)	O(log n)	O(log n)	Yes
Hash table	**O(1) avg**	O(1) avg	O(1) avg	No

5.8 Quick Revision & Practice

ONE-PAGE SUMMARY

- Tree: hierarchical, n nodes → n − 1 edges; binary tree ≤ 2 children.
- BST: left < node < right; inorder = sorted; search / insert / delete O(h).
- Delete: leaf → remove; one child → bypass; two children → inorder successor.
- Traversals: In = L N R, Pre = N L R, Post = L R N, Level = queue.
- Hashing: index = h(key); collision = two keys, same slot; load factor λ = n / m.
- Chaining = lists in slots; open addressing = probe (linear, quadratic, double).

Likely exam questions

2 marks

- Define binary search tree. Define root, leaf and height.
- What is a collision? What is load factor?
- Write the inorder traversal of a given BST.
- Differentiate chaining and open addressing.

5 marks

- Construct a BST from 45, 20, 60, 10, 30, 55, 70 and list all traversals.
- Explain deletion of a node with two children in a BST.
- Explain any three hash functions with examples.
- Insert 12, 18, 25, 32, 5 into a table of size 7 using linear probing.

10 marks

- Explain BST insertion, deletion and traversals with algorithms and diagrams.
- Explain hashing, collision resolution techniques and applications with examples.

Try it yourself

1. Insert **8, 3, 10, 1, 6, 14, 4, 7, 13** into an empty BST. Write the inorder, preorder and postorder sequences. *(Inorder: 1 3 4 6 7 8 10 13 14)*
2. Delete 3 from the tree above (it has two children). Which node replaces it? *(4 – inorder successor)*
3. Using h(k) = k mod 10 and chaining, store 31, 41, 51, 12, 22. Draw the table.
4. What is the height of a BST built from 1, 2, 3, 4, 5 inserted in this order? *(4 – skewed)*